

Python for Data and Machine Learning

A visual, example-driven tour of the scientific Python stack

Jonghwan Lee

2026

Contents

Preface	11
Who this book is for	11
How to read this book	11
Conventions	12
A note on reproducibility	12
Acknowledgements	13
 Why Python for Data	 13
1.1 A language for data	13
1.2 The four libraries that do most of the work	14
1.3 A first end-to-end example	15
1.4 What isn't in the stack	20
1.5 How this book is organized	21
1.6 Setup sanity check	22
1.7 Exercises	22
 The Working Environment	 23
2.1 Which Python	23
2.2 Virtual environments: pick one tool	24
2.3 Where your code lives	26
2.4 What `import` actually does	27
2.5 A reproducible starter project	28
2.6 Running the code in this book	30
2.7 Exercises	30

Core Python, Fast **31**

3.1 Numbers, strings, and the things that are truthy	31
3.2 Data structures you will use constantly	32
3.3 Control flow, briefly	34
3.4 Functions, arguments, and type hints	35
3.5 Files, paths, and reading data	37
3.6 Errors and exceptions	38
3.7 Classes, only as far as you need	38
3.8 A data-cleaning warm-up	40
3.9 Exercises	41

NumPy Arrays **42**

4.1 What an ndarray is	42
4.2 Creating arrays	43
4.3 dtypes	44
4.4 Indexing and slicing	46
4.5 Views vs copies	47
4.6 Reshaping and transposing	48
4.7 Aggregations	49
4.8 Saving and loading	49
4.9 Exercises	50

Vectorization and Broadcasting **50**

5.1 Why loops are slow, visibly	51
5.2 Universal functions	52
5.3 Broadcasting	52

5.4 Vectorizing common idioms	54
5.5 Reshaping for broadcasting	56
5.6 When vectorization is hard	56
5.7 Exercises	57
Linear Algebra and Random Numbers	58
6.1 Matrix multiplication and `@`	58
6.2 Solving linear systems	59
6.3 The other decompositions	59
6.4 Randomness done right	60
6.5 Monte Carlo: putting it together	61
6.6 Exercises	63
Series and DataFrame	63
7.1 The Series	64
7.2 The DataFrame	64
7.3 Selecting a column	66
7.4 Selecting rows	66
7.5 Reading and writing files	67
7.6 Dtypes that aren't NumPy	69
7.7 Memory	69
7.8 A quick tour of what's next	70
7.9 Exercises	70
Selecting, Filtering, Cleaning	71
8.1 `loc` vs `iloc`, properly	71
8.2 The `SettingWithCopyWarning`	72

8.3 Missing data	73
8.4 Type coercion	74
8.5 String methods	74
8.6 Datetime methods	74
8.7 `assign` and method chaining	75
8.8 A small cleaning worked example	76
8.9 Exercises	77

Grouping, Joining, Reshaping 78

9.1 Split–apply–combine	78
9.2 Joining: `merge` vs `concat`	81
9.3 Reshape: long vs wide	82
9.4 Rolling windows	83
9.5 Time series resampling	84
9.6 A full example	84
9.7 Exercises	85

matplotlib Fundamentals 86

10.1 Figure, Axes, Artist	86
10.2 Grids of Axes	88
10.3 Plot types you'll use constantly	89
10.4 Colors, styles, and rcParams	90
10.5 Saving for print vs. screen	90
10.6 The closest thing to a universal plot idiom	91
10.7 Exercises	91

Statistical Plots 92

11.1 Histograms	93
11.2 KDE — kernel density estimates	94
11.3 Box plots	95
11.4 Violin plots	95
11.5 Scatter with error	95
11.6 Scatter matrices	96
11.7 Choosing a plot	96
11.8 Exercises	97

Designing Figures that Communicate 97

12.1 The four things readers see first	98
12.2 Before and after: a reusable style	98
12.3 Choose axes before you draw	99
12.4 Annotate the point	100
12.5 Small multiples	101
12.6 Color, for the reader (not for the author)	101
12.7 Saving for the medium	102
12.8 Exercises	102

Statistics in Code 103

13.1 Descriptive statistics	103
13.2 Distributions from <code>scipy.stats</code>	104
13.3 Confidence intervals, two ways	105
13.4 Hypothesis tests, demystified	106
13.5 Multiple testing	108
13.6 A note on practical significance	108
13.7 Exercises	108

The scikit-learn Interface 109

14.1 Estimators, transformers, and predictors	109
14.2 `fit_transform` and why it's a lie	110
14.3 Pipelines	111
14.4 Training, validation, testing	111
14.5 `random_state` and reproducibility	112
14.6 Column-by-column preprocessing	112
14.7 Custom estimators (briefly)	113
14.8 Exercises	114

Regression 114

15.1 Linear regression by hand	114
15.2 Linear regression with scikit-learn	115
15.3 Evaluating a regressor	115
15.4 Regularization	116
15.5 Nonlinear features, linearly modelled	118
15.6 Diagnostics: residuals	118
15.7 Beyond linear: trees, boosting	119
15.8 Exercises	120

Classification 120

16.1 Logistic regression	120
16.2 Decision trees and random forests	121
16.3 Gradient boosting	122
16.4 Metrics beyond accuracy	123
16.5 ROC and PR curves	124

16.6 Class imbalance	124
16.7 Probability calibration	125
16.8 Exercises	125

Evaluation, Selection, Tuning 126

17.1 Cross-validation, properly	126
17.2 Grid search vs. random search	127
17.3 Nested cross-validation	128
17.4 Learning curves	129
17.5 Leakage, the sneaky kind	130
17.6 Feature importance, again	130
17.7 Reporting results, honestly	131
17.8 Exercises	131

An End-to-End Project 132

18.1 Load and look	132
18.2 Exploratory figures	133
18.3 A train / test split	134
18.4 A baseline	134
18.5 A linear baseline	135
18.6 Adding nonlinear features	135
18.7 A gradient-boosted fit	136
18.8 Evaluate on the held-out test set	137
18.9 A report figure	137
18.10 What this chapter didn't do	138
18.11 Exercises	139

Productionizing Notebooks**139**

19.1 The notebook-to-script journey	139
19.2 Logging, not print	141
19.3 Pinning versions	142
19.4 Packaging a model for serving	143
19.5 Monitoring	144
19.6 When to stop using a notebook	144
19.7 Where to go next	144
19.8 Exercises	145

Appendix A: Environment Cheat Sheet**146**

A.1 venv	146
A.2 pip	146
A.3 uv	147
A.4 Jupyter	147
A.5 Project layout	148

Appendix B: API Index**149**

B.1 NumPy	149
B.2 pandas	150
B.3 matplotlib	150
B.4 scikit-learn	151

Appendix C: Further Reading**152**

C.1 Python the language	152
C.2 NumPy and the array world	152
C.3 pandas, beyond the basics	153

C.4 Visualization	153
C.5 Statistics	153
C.6 Machine learning	153
C.7 Deep learning	154
C.8 Engineering the surrounding code	154
C.9 When you want to stop reading books	154

CHAPTER

Preface

This book teaches Python the way I wish it had been taught to me when I moved from another language into data work: not as a grammar lesson, but as a tour of the small number of tools that, taken together, let you go from a CSV file on disk to a working model to a figure in a report.

The scientific Python stack is remarkably compact. Four libraries — NumPy, pandas, matplotlib, and scikit-learn — cover the large majority of what a practising data analyst or ML engineer does day to day. Each of them has its own idioms, its own sharp edges, and its own moments of quiet brilliance. This book is a guided tour of all four, strung together on a spine of ordinary Python, with worked examples end to end.

Who this book is for

You should be comfortable with at least one programming language. That language doesn't have to be Python — Chapter 3 is a concentrated crash course on the pieces of the language you actually need. If you have never programmed before, this is probably not the right starting point.

You don't need to know anything about machine learning. Part V develops the ideas as we use them, with just enough statistics to read the outputs honestly.

How to read this book

Every chapter follows the same shape:

1. **Idea** — what the chapter is about and why it matters, in prose.
2. **Worked examples** — code blocks paired with their output. Nothing is hidden; everything you see is runnable.
3. **Figures** — generated from the code in the previous section, never decorative.
4. **Exercises** — small, specific, usually solvable in under 20 minutes.

The examples build on one another within a chapter but rarely across chapters, so skipping around is fine. If you read linearly, the cumulative effect is that by Chapter 18 you're writing something that looks a lot like a real analysis notebook without thinking about it.

Conventions

Code that you type appears in a monospaced box:

```
import numpy as np

rng = np.random.default_rng(seed=0)
x = rng.standard_normal(5)
print(x)
```

Its output, when shown, follows immediately below:

```
[ 0.12573022 -0.13210486  0.64042265  0.10490012 -0.53566937]
```

Three kinds of inline callouts appear throughout the book:

Note

A clarification, a cross-reference, or a digression that earns its keep.

Tip

A concrete suggestion — something I've learned to do (or avoid) on real projects.

Warning

A place where the obvious reading is wrong, and the bug is easy to ship.

A note on reproducibility

Every figure in this book is produced by a Python script that lives alongside the manuscript. You can regenerate any of them by running `make figures`. Every random seed is fixed, so the numbers you see in the book are the numbers you'll see on your laptop. Where something depends on library version, I've noted the versions I used.

Acknowledgements

To the people who read early drafts and told me when the explanations made no sense: thank you. The bugs that remain are mine.

CHAPTER

Why Python for Data

Python became the default language of data work by accident. It was designed in 1991 as a general-purpose scripting language with a readable syntax and no particular numerical ambitions. The reason it now sits underneath most of the world's statistics, analytics, and machine-learning code is that four libraries — written by four different groups, over twenty-five years — quietly turned it into something it was never meant to be: a glue language for fast numerical arrays, and therefore a glue language for data.

This chapter is a tour of that accident. By the end of it you will have loaded a real dataset, looked at it, trained a small model on it, and drawn a figure that shows what the model learned. Every line of code you see is runnable on a laptop in a few seconds, and every figure in this chapter is produced by a script that ships with the book. We will move quickly; later chapters come back to every piece in detail.

1.1 A language for data

The practical question is not "is Python fast?" — it isn't, especially, compared to C or Rust — but "is Python fast *enough* when the expensive loops are written in C underneath?" The answer has turned out to be a resounding yes. Almost every operation on a large array in scientific Python is a thin wrapper over a compiled routine, and the Python layer exists mostly to let you describe *which* compiled routine to call and on *what*.

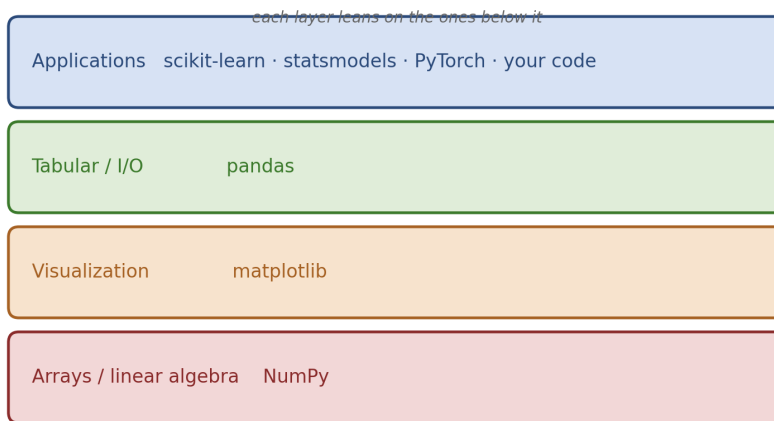
That division of labour is what makes the language a good fit for data. You get to write the part that is about your problem — "standardize these columns, then fit a model to the result" — in a language that reads like English, while the part that is about floating-point throughput is handled by libraries you never need to open.

Note

When people say "Python is slow," they usually mean that a pure-Python `for` loop over ten million numbers is slow. They are right. The response is almost always to not write that loop in the first place — Chapter 5 is devoted to the alternative.

1.2 The four libraries that do most of the work

The scientific Python stack



The layers of the scientific Python stack. Each layer uses the layers below it; you rarely touch NumPy directly once you're working in pandas, but it is always underneath.

The four libraries that do most of the work are:

NumPy. An `ndarray` type that stores numbers in a contiguous block of memory, together with a large set of fast operations over those arrays. Everything else in the stack sits on top of it. When a pandas `Series` adds two columns, or matplotlib draws a curve, or scikit-learn multiplies a weight matrix by a batch of features, the real work is done in NumPy.

pandas. A `DataFrame` type that is essentially a dictionary of NumPy arrays with a shared index. It adds everything you need to treat tabular data as tabular data: labelled rows and columns, missing values, joins, group-by, and reading and writing almost every format you will encounter in the wild.

matplotlib. A plotting library that is old, enormous, occasionally awkward, and utterly dominant. If you need a figure to look a specific way, matplotlib can do it; the price is that you sometimes have to tell it exactly what you want.

scikit-learn. A consistent interface to most of the models a practising data scientist uses in a week — linear models, trees, random forests, gradient boosting, clustering, dimensionality reduction, and the plumbing (cross-validation, pipelines, metrics) that holds them together.

These are not the only libraries you will ever use. Over the course of a project you will probably also reach for `scipy` (for distributions and signal processing), `statsmodels` (for classical statistics that scikit-learn doesn't bother with), and one of the deep-learning frameworks — PyTorch for most new work, JAX when you want the compiler to do more for you. But the four libraries above are the ones you will touch every day, and they are the ones this book is about.

1.3 A first end-to-end example

The goal of this section is to take you through every stage of a real analysis in less than a page of code: load a dataset, explore it, train a model, and look at what it learned. The dataset is the iris flower dataset — 150 specimens, four measurements each, three species. It is small, clean, and ships with scikit-learn, which makes it perfect for a first pass. We will meet harder data in Chapter 7.

Loading the data

```
import pandas as pd
from sklearn.datasets import load_iris

raw = load_iris(as_frame=True)
df = raw.frame.rename(columns={"target": "species"})
df["species"] = df["species"].map(dict(enumerate(raw.target_names)))

print(df.shape)
print(df.head())
```

```
(150, 5)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
0                5.1                3.5                1.4                0.2  setosa
1                4.9                3.0                1.4                0.2  setosa
2                4.7                3.2                1.3                0.2  setosa
3                4.6                3.1                1.5                0.2  setosa
4                5.0                3.6                1.4                0.2  setosa
```

Four lines of code, and we have a 150-row, five-column DataFrame with the species names spelled out. Three things worth noticing:

1. `load_iris(as_frame=True)` returns a `Bunch` object — a namespace with a `.frame` attribute, a `.data` array, a `.target` array, and some metadata. scikit-learn uses this shape for all of its toy datasets.
2. `df.rename(columns=...)` returns a new DataFrame; it does not modify `raw.frame` in place. This is the pandas default and we will rely on it constantly.
3. The `.map(...)` call turns integer codes (0, 1, 2) into string labels (`setosa` , `versicolor` , `virginica`). Mapping through a dictionary is how you convert between representations throughout pandas.

A first look

Before fitting anything, you should always look at the numbers. Two summaries will do:

```
print(df.drop(columns="species").describe().round(2))
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
count            150.00            150.00            150.00            150.00
mean              5.84              3.06              3.76              1.20
std              0.83              0.44              1.77              0.76
min              4.30              2.00              1.00              0.10
25%              5.10              2.80              1.60              0.30
50%              5.80              3.00              4.35              1.30
75%              6.40              3.30              5.10              1.80
max              7.90              4.40              6.90              2.50
```

`describe()` is a constant companion during exploratory work. It gives you the eight numbers per column that tell you whether anything is obviously wrong: a wrong unit (petal lengths of 1000), a broken scale (widths that are all identical), a stray sentinel (`-999`)

where a missing value should be). Nothing looks wrong here.

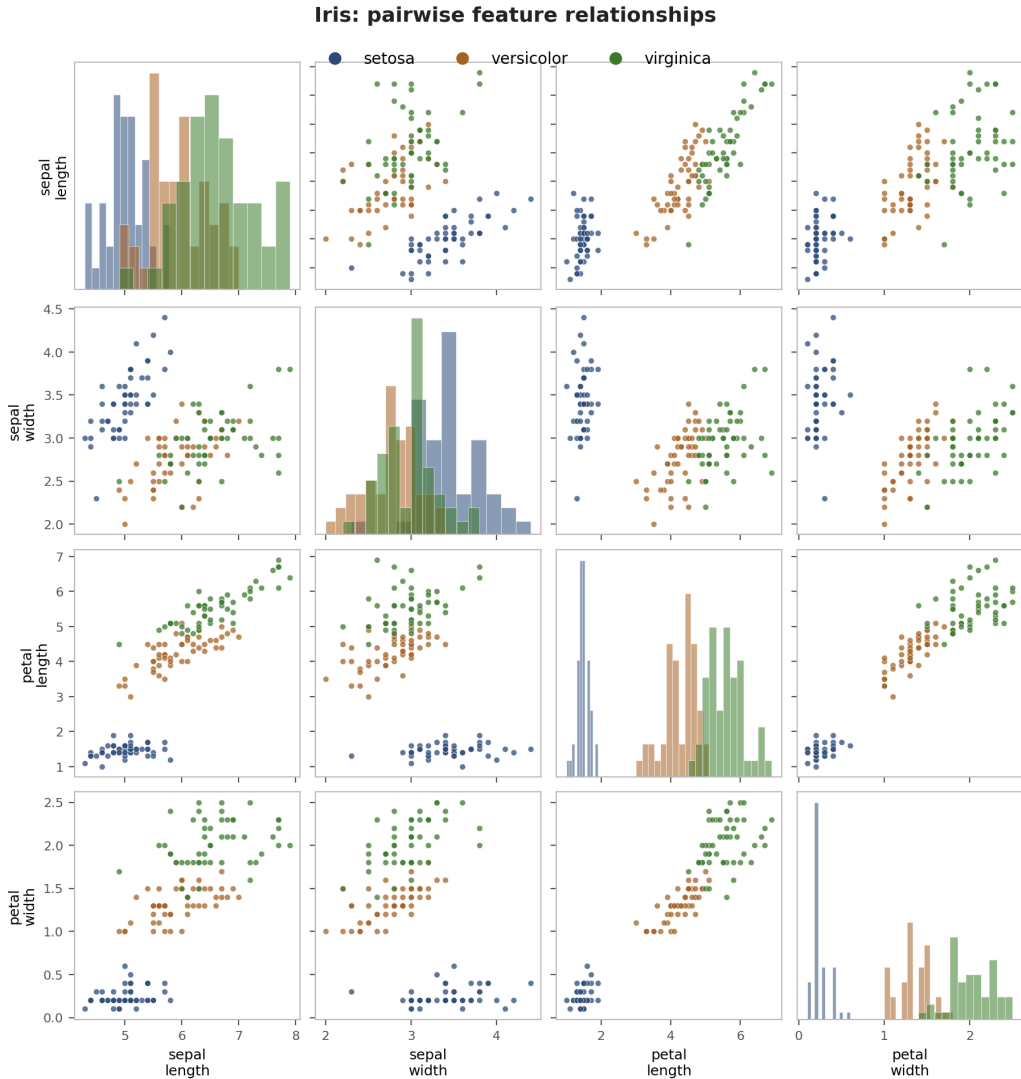
More interesting is the per-species breakdown:

```
print(df.groupby("species").mean().round(2))
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
species				
setosa	5.01	3.43	1.46	0.25
versicolor	5.94	2.77	4.26	1.33
virginica	6.59	2.97	5.55	2.03

Two observations jump out. First, petal length and petal width are dramatically different between species — setosa's petals are three to four times smaller than virginica's. Sepals are less informative. Second, setosa looks like it should be easy to separate from the other two, and virginica and versicolor look like they will overlap a little.

You can see both observations at once in a scatter matrix, which is the first figure we will produce. The scatter matrix puts every feature against every other feature on the off-diagonal, and a histogram of each feature on the diagonal — a cheap way to see the univariate and bivariate structure of a small dataset at once.



Iris feature pairs. The diagonal shows the distribution of each feature per species; the off-diagonal cells show every pair of features as a scatter plot. Setosa (blue) is separable from the other two in almost every petal-based projection; versicolor and virginica overlap slightly along the petal axes and heavily along the sepal axes.

This figure is one matplotlib script, not a loop over a dozen `plt.scatter` calls — but it's built out of a loop over a dozen `plt.scatter` calls, in a 4-by-4 grid. Chapter 10 shows you how to assemble grids like this cleanly.

Fitting a model

Looking at the scatter matrix, the two petal features alone seem to separate the species. Let's train a logistic-regression classifier on just those two features, so we can visualize what the model decides in two dimensions.

```
from sklearn.linear_model import LogisticRegression

X = df[["petal length (cm)", "petal width (cm)"]].to_numpy()
y = df["species"].to_numpy()

model = LogisticRegression(max_iter=500).fit(X, y)
print(f"training accuracy: {model.score(X, y):.2%}")
```

```
training accuracy: 96.67%
```

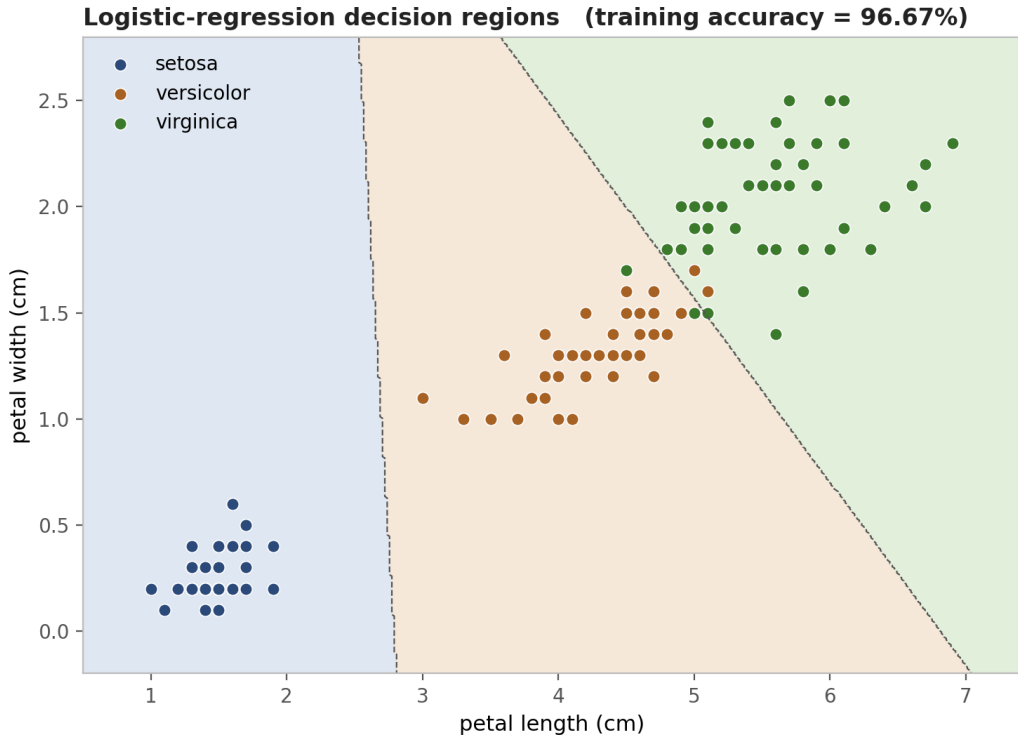
That is the scikit-learn pattern you will use for nearly every model in this book: `fit(X, y)` on the training data, `score(X, y)` to evaluate, `predict(X_new)` to use the model. The same three lines, in the same order, work whether the model is a logistic regression, a random forest, or a gradient-boosted ensemble. Chapter 14 explains why this uniformity is the single most important idea in scikit-learn.

Warning

Training accuracy is almost never what you care about. It measures how well the model fits the data it has already seen, and any sufficiently flexible model can drive it to 100%. In Chapter 14 we'll split the data into training and validation sets; for this chapter, the point is just that the model converged and learned something.

Looking at what the model learned

A classifier with two input features has a natural visualization: the decision regions it induces on the feature plane. We build a dense grid of (petal length, petal width) pairs, ask the model to classify every one of them, and shade the plane by the predicted class. The training points go on top.



What the trained logistic regression does to the petal-length / petal-width plane. The dashed lines are the two decision boundaries; the shaded regions are the classes the model predicts. Almost every training point falls in the region of its correct class — the handful of misclassifications are on the versicolor/virginica border.

You can read the model off this figure. It has learned three (almost) linearly separable regions. Setosa is cleanly cut off by a diagonal in the lower left; virginica occupies the upper right; versicolor sits between them. The few stray points on the boundary between versicolor and virginica are responsible for the 3.3% error rate.

The thing to notice is that the *entire* workflow — load, inspect, visualize, fit, evaluate, visualize again — fit in under 30 lines of code, spread across four libraries that were not originally designed to work together. That is what we mean by "Python for data": a small, boring, predictable vocabulary that lets you move quickly from a CSV file to a figure you can put in a report.

1.4 What isn't in the stack

It is worth being honest about what Python does not do well.

Concurrency. The global interpreter lock and the overhead of process-based parallelism mean that anything CPU-bound and not already vectorized is painful to scale. The scientific stack routes around this by pushing the work into C; when you can't, you reach for `multiprocessing`, `joblib`, `dask`, or `ray`, and each of them has rough edges.

Deploying a model. Training a model in Python is easy. Serving it at low latency, with version control and monitoring, is no easier in Python than in any other language and is sometimes harder, because the dependency graph of a scientific project tends to be heavy. Chapter 19 is about closing this gap.

Strict type safety. Python's type hints are checked by external tools (`mypy`, `pyright`) and not enforced by the runtime. For large codebases this is a real tax. The data-science community has increasingly adopted type hints for library code, and I will use them throughout the book, but they remain a suggestion, not a guarantee.

None of these are dealbreakers for most data work. They are worth knowing so you know when to step outside the stack.

1.5 How this book is organized

The book is six parts.

Part I (Chapters 1–3) is foundations: this tour chapter, a setup chapter, and a concentrated crash course in the parts of the Python language you actually need. If you already know Python, you can read Part I in an afternoon.

Part II (Chapters 4–6) is NumPy, top to bottom: arrays, broadcasting, linear algebra, randomness. Everything else in the stack sits on this foundation.

Part III (Chapters 7–9) is pandas: the DataFrame, indexing and selection, cleaning, group-by, joins, time series. This is the part most analysts spend most of their time in.

Part IV (Chapters 10–12) is matplotlib and figure design — first the mechanics, then the judgement calls.

Part V (Chapters 13–17) is statistics and scikit-learn: from the `fit` / `predict` interface through regression, classification, and model evaluation.

Part VI (Chapters 18–19) pulls it all together in an end-to-end worked project and a short chapter on turning notebooks into production code.

You can read linearly, or you can skip to whichever part answers a question you have right now. The chapters cross-reference each other, but each one is written to stand alone.

1.6 Setup sanity check

Before you move on, make sure the following runs without error. It's the smallest possible version of the workflow we just walked through:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
X = rng.standard_normal((200, 2))
y = (X[:, 0] + X[:, 1] > 0).astype(int)

model = LogisticRegression().fit(X, y)
print(f"accuracy: {model.score(X, y):.2%}")

fig, ax = plt.subplots()
ax.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", edgecolor="white")
ax.set_title("Setup check: two Gaussian clouds")
fig.savefig("setup_check.png", dpi=150)
```

If it prints something like `accuracy: 98.50%` and writes `setup_check.png` to your working directory, every library the book needs is installed and working. If anything fails, jump to Chapter 2 — it's about getting your environment into a shape where the rest of this book will work the first time.

1.7 Exercises

1. Change the classifier in Section 1.3 from `LogisticRegression` to `KNeighborsClassifier(n_neighbors=5)`. What happens to the training accuracy? What happens to the shape of the decision boundary? (You will need `from sklearn.neighbors import KNeighborsClassifier`.)
2. In the same example, use all four features — sepal length, sepal width, petal length, petal width — instead of just the two petal features. What training accuracy do you get? Can you still draw a decision-boundary figure? Why or why not?

3. Reproduce the per-species mean table in Section 1.3, but also include the standard deviation of each feature per species. Hint: `df.groupby("species").agg(["mean", "std"])` .
4. Open `scripts/figures/ch01_iris_scatter.py` and change the colour palette. Rerun `make figures` . What was the easiest part of this task, and what surprised you? Keep the answer in mind — Chapter 12 is about exactly this problem.

In Chapter 2 we'll set up an environment that makes the code in this book reproducible on your machine. Then, in Chapter 3, we'll walk through the subset of Python the rest of the book assumes you know — and no more.

CHAPTER

The Working Environment

The single most common reason someone bounces off a technical book is that the setup doesn't work. You install Python, `import numpy` fails in one window but succeeds in another, and by the time you have figured out why you have lost an afternoon and some of your will to live. This chapter is an attempt to prevent that.

By the end of it you should have: one Python version you trust, one way to make a project-local environment, one way to start a Jupyter notebook, and a mental model of what happens when you type `import pandas` .

2.1 Which Python

There is no such thing as "Python" on a modern laptop; there are several, each of which may answer to the command `python` or `python3` . Before anything else, find out what you actually have:

```
$ which -a python python3
/usr/bin/python3
/opt/homebrew/bin/python3

$ python3 --version
Python 3.11.7

$ python3 -c "import sys; print(sys.prefix)"
/opt/homebrew/opt/python@3.11
```

Three things worth understanding from that output:

1. **There are multiple Pythons on your** `$PATH`. The first one wins. If your shell finds `/usr/bin/python3` first, that is a *system* Python — the one Apple or your Linux distribution ships with and uses for its own scripts. You should not install packages into it.
2. **The interpreter at** `sys.prefix` **is the installation root.** Every `site-packages` directory lives under there, which means two Pythons have two separate package sets. This is why `numpy` is sometimes missing in one terminal and present in another.
3. **Version matters.** The examples in this book run on Python 3.11 and later. Anything older is end-of-life for the libraries we care about. If `python3 --version` says 3.10 or below, you want to install a newer one.

Tip

Prefer a Python you installed yourself (via `pyenv`, `uv`, Homebrew, or the official installer) over the system Python. The system Python is for your operating system, not for you.

2.2 Virtual environments: pick one tool

A *virtual environment* is a directory that contains its own Python interpreter symlink and its own `site-packages`. Activate it and `pip install` writes into that directory instead of polluting a global site. Every serious project should have one.

There are five common ways to make one. This is overkill. You only need to know one:

Tool

One-line intro

When to use

`venv`

Ships with Python. `python -m venv .venv`

Default. Always works.

`uv`

A single fast Rust binary that replaces `venv` + `pip`

New projects, fastest install

`conda`

Manages binary packages (e.g. CUDA builds) from Anaconda

GPU / scientific binaries on Windows

`poetry`

`pyproject.toml` -driven project manager

Publishing a library to PyPI

`pipenv`

Older tool in the poetry niche

Legacy projects only

For this book, I'll use `venv` + `pip` in examples because they are guaranteed to be present. If you want a faster workflow, `uv` is a drop-in replacement that runs an order of magnitude quicker.

The minimum workflow

```
$ cd my-project
$ python3 -m venv .venv
$ source .venv/bin/activate          # Windows: .venv\Scripts\activate
(.venv) $ python -m pip install --upgrade pip
(.venv) $ python -m pip install numpy pandas matplotlib scikit-learn jupyterlab
(.venv) $ python -c "import numpy; print(numpy.__version__)"
2.1.3
```

Notice the prompt changed to `(.venv)` — that's the visible indication that an environment is active. `which python` now points inside `./venv/bin/`, not into a global install.

To freeze the exact versions you're using so someone else can reproduce the environment:

```
(.venv) $ python -m pip freeze > requirements.txt
```

And to leave the environment:

```
(.venv) $ deactivate
```

Warning

Do not check `.venv/` into git. It is large, platform-specific, and every collaborator will need their own. Commit `requirements.txt` (or a lock file) instead. A single line in `.gitignore` takes care of it:

```
.venv/
```

The uv shortcut

`uv` bundles virtual-env creation, dependency resolution, and package installation into one binary. The same workflow in `uv` is three commands:

```
$ uv venv                                # creates .venv
$ source .venv/bin/activate
(.venv) $ uv pip install numpy pandas matplotlib scikit-learn jupyterlab
```

The install step is typically 10–20× faster than `pip`'s because it uses a pre-resolved package index and a global cache shared across projects. The output is identical — you still end up with a `.venv` full of the same wheels.

2.3 Where your code lives

You have two reasonable options for where to write Python:

Notebooks. JupyterLab opens in a browser and gives you cells you can run out of order. This is the right environment for exploratory work — you load data, poke at it, keep what works, and throw out the rest. Every figure in this book was produced from a notebook first and promoted into a script once it stabilized.

```
(.venv) $ jupyter lab
```

Editors. For code that needs to run unsupervised — a data pipeline, a utility module, a test — use an editor. VS Code and PyCharm both have good Python support. VS Code in particular supports notebooks inline, so you can use one tool for both modes.

I'll show code throughout the book as if it's in a notebook — cells of code followed by their output. Everything translates to a script unchanged; if you save a file called `foo.py` and run `python foo.py`, the same code does the same thing, it just doesn't print the final expression of each cell automatically.

The REPL habit

Even with notebooks everywhere, the plain Python REPL still has a place. When you want to check the type of something, remember what arguments a function takes, or confirm that your environment can actually import something, the three-line REPL session is faster than anything else:

```
(.venv) $ python
>>> import numpy as np
>>> x = np.array([1, 2, 3])
>>> x.dtype
dtype('int64')
>>> np.mean?
Type:          function
...
```

The `?` suffix (in IPython and Jupyter) shows the docstring; `??` shows the source. Both are faster than alt-tabbing to documentation.

2.4 What `import` actually does

Something that will save you hours, eventually, is a rough mental model of how `import numpy` works.

1. Python looks at `sys.path`, which is a list of directories.
2. It searches each one for a package named `numpy` (a directory with `__init__.py`) or a module named `numpy.py`.
3. The first match wins. It runs the module's top-level code, attaches the result to the `numpy` name in your namespace, and stores it in `sys.modules` so subsequent imports are free.

You can see the search path:

```
import sys
for p in sys.path[:5]:
    print(p)
```

```
/Users/me/my-project/.venv/lib/python3.11/site-packages
/opt/homebrew/opt/python@3.11/Frameworks/Python.framework/Versions/3.11/lib/python31
...
```

When things go wrong, it's almost always because `sys.path` points somewhere you didn't expect. The first entry is your virtual env's `site-packages`, which is correct. If it isn't — if you see a global or system `site-packages` first — your env isn't activated.

Note

`pip` and `python` must agree on which Python they're acting on. That is why every command in this chapter is `python -m pip install`, never just `pip install`: the `-m` makes Python run `pip` *from the active interpreter*, so there's no ambiguity. This is the single best habit you can pick up from this chapter.

2.5 A reproducible starter project

Here is a layout I use for every new project. Copy-paste it and move on.

```

my-project/
├── .gitignore
├── README.md
├── requirements.txt          # or pyproject.toml
├── data/                    # raw inputs, gitignored
├── notebooks/               # .ipynb files
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   ├── clean.py         # reusable functions
│   └── tests/
│       └── test_clean.py

```

The rules:

- `data/` is gitignored. Raw data shouldn't live in git — it's too big, it might be sensitive, and it belongs with the cloud bucket it came from. Keep a pointer in README instead.
- `notebooks/` is fair game for experimentation, but anything you depend on twice moves into `src/`.
- `src/my_project/` is a real Python package you can `pip install -e .` and then from `my_project.clean import ...` from notebooks, scripts, and tests alike.

A minimal `pyproject.toml` for that package:

```

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "my-project"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "numpy>=1.26",
    "pandas>=2.1",
    "matplotlib>=3.8",
    "scikit-learn>=1.4",
]

```

2.6 Running the code in this book

Clone the book repo, make a virtual environment, install the dependencies, and you're set:

```
$ git clone https://example.invalid/python-for-data-ml.git
$ cd python-for-data-ml
$ python3 -m venv .venv && source .venv/bin/activate
(.venv) $ python -m pip install -r requirements.txt
(.venv) $ jupyter lab notebooks/
```

Every code listing in every chapter is runnable as-is. Where the output depends on a file, the file is in `data/`. Where it depends on a seed, the seed is fixed in the code. If you run a cell and get different numbers than the book shows, one of two things is usually true: your library versions are much newer, or you have not activated the environment. Check `which python` first.

2.7 Exercises

1. Run `python -m pip list` in your activated environment. How many packages are installed? Which of them did you install directly, and which were pulled in as dependencies? (Hint: `python -m pip list --not-required .`)
2. Break your environment on purpose: inside the activated venv, run `pip install "numpy==1.19"`. What error does `import numpy` give you? What does the `pandas` import error say about it? Fix it with `pip install --upgrade numpy`.
3. Run the following in two fresh terminals — one with the env activated, one without:

```
import sys
print(sys.path[0])
```

What's different? Why?

4. Read the output of `python -m pip install --dry-run scikit-learn` without actually installing anything. What else would it install, and why?

Chapter 3 is a fast tour of the Python language itself — just the pieces you actually need for the rest of this book.

CHAPTER

Core Python, Fast

If you have written Python before, skim this chapter looking for anything that surprises you, and move on. If you have written another language but not Python, read it straight through. If you have never written code at all — this book is probably not the right entry point; come back after a beginner's Python tutorial and the rest will click.

My goal in this chapter is narrow: cover the subset of the Python language that the rest of the book assumes, and no more. Generators, metaclasses, and `__slots__` are real and sometimes wonderful, but they are not on the critical path for data work.

3.1 Numbers, strings, and the things that are truthy

```
x = 3          # int
pi = 3.14159   # float
hello = "world" # str
flag = True    # bool — also an int, oddly enough
nothing = None # the sole value of NoneType
```

Three things worth pinning down early.

Integers have arbitrary precision. `2 ** 200` is not an overflow; it's a number.

```
>>> 2 ** 200
1606938044258990275541962092341162602522202993782792835301376
```

Floats are IEEE-754 doubles. They do not exactly represent most decimal fractions, and you should never compare them with `==`:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
```

Strings are Unicode. They have a rich method set and you should reach for those methods before regular expressions:

```
>>> " 2026-04-18 ".strip().split("-")
['2026', '04', '18']
>>> f"pi to 2dp: {pi:.2f}"
'pi to 2dp: 3.14'
```

The f-string is how you format text. `{expr: spec}` — the expression on the left, a format spec on the right — is the modern idiom. You will see the older `"%s" % x` form in old code; prefer f-strings for anything new.

Truthiness

Any Python value can be used in a boolean context. The falsy values are a short list:

```
falsy = [False, None, 0, 0.0, "", [], {}, set(), ()]
for x in falsy:
    assert not x
```

Everything else is truthy. This is useful — `if items:` is idiomatic for "if this list is non-empty" — and it is occasionally a trap, because `0` and `"0"` behave differently:

```
>>> bool(0)
False
>>> bool("0")
True
```

3.2 Data structures you will use constantly

Four container types do 95% of the work:


```
# list: ordered, mutable, inhomogeneous
xs = [1, 2, 3]
xs.append(4)

# tuple: ordered, immutable, often inhomogeneous
point = (3.0, 4.0)
x, y = point          # unpacking

# dict: mapping from hashable keys to values, insertion-ordered
meta = {"author": "JL", "year": 2026}
meta["year"]          # 2026

# set: unordered collection of unique hashable values
tags = {"python", "data", "python"}
# -> {"data", "python"}
```

Two idioms worth internalizing.

Indexing and slicing

Python indexing is zero-based and slicing is `start:stop:step` with `stop` exclusive. Negative indices count from the right.

```
letters = ["a", "b", "c", "d", "e"]
letters[0]      # 'a'
letters[-1]     # 'e' (last)
letters[1:4]    # ['b', 'c', 'd'] (stop is exclusive)
letters[:2]     # ['a', 'b']
letters[::2]    # ['a', 'c', 'e'] (every second element)
letters[::-1]   # ['e', 'd', 'c', 'b', 'a'] (reversed)
```

The same slicing syntax works on tuples and strings, and — this is the point — it generalizes to NumPy arrays and pandas Series. Every slice you learn here is a slice you will write in a DataFrame.

Comprehensions

A list comprehension is a compact `for` loop that builds a list. Comprehensions are the single most important Python idiom to get fluent in, because they make data-shaping code read top-to-bottom:

```
# Squares of 0..9
squares = [n * n for n in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Even squares only
evens = [n * n for n in range(10) if n % 2 == 0]
# [0, 4, 16, 36, 64]

# Dict comprehension: map word -> length
words = ["apple", "banana", "cherry"]
lengths = {w: len(w) for w in words}
# {'apple': 5, 'banana': 6, 'cherry': 6}

# Set comprehension: unique first letters
firsts = {w[0] for w in words}
# {'a', 'b', 'c'}
```

Tip

If your comprehension grows past two clauses or eighty columns, un-roll it into a `for` loop. A comprehension is only a good idea when it is shorter and clearer than the loop.

3.3 Control flow, briefly

```

# Conditional
if x > 0:
    sign = "pos"
elif x < 0:
    sign = "neg"
else:
    sign = "zero"

# While
n = 10
while n > 0:
    n -= 1

# For – always iterates over something; there is no C-style for loop.
for name in ["alice", "bob", "carol"]:
    print(name)

# enumerate gives you (index, value) pairs
for i, name in enumerate(["alice", "bob", "carol"]):
    print(i, name)

# zip walks multiple iterables in lockstep
for name, age in zip(["alice", "bob"], [30, 35]):
    print(name, age)

```

`range(n)` produces the integers 0, 1, ..., n-1 . `range(a, b)` is a, a+1, ..., b-1 . `range(a, b, step)` is what you'd guess. None of these allocate a list — `range` is lazy — which means `for i in range(10**9):` is fine memory-wise (though obviously slow).

3.4 Functions, arguments, and type hints

A function definition:

```
def standardize(xs: list[float]) -> list[float]:
    """Return xs rescaled to zero mean, unit variance."""
    mean = sum(xs) / len(xs)
    var = sum((x - mean) ** 2 for x in xs) / len(xs)
    sd = var ** 0.5
    return [(x - mean) / sd for x in xs]

>>> standardize([1, 2, 3, 4, 5])
[-1.414..., -0.707..., 0.0, 0.707..., 1.414...]
```

Four things to notice:

1. **Default arguments** are evaluated once, at function-definition time. This is a trap:

```
def add(item, into=[]):      # DO NOT do this
    into.append(item)
    return into
```

Every call without `into=` shares the *same* list. Use `None` instead:

```
def add(item, into=None):
    if into is None:
        into = []
    into.append(item)
    return into
```

2. **Keyword arguments** let the caller name the argument:

```
def plot(x, y, *, color="blue", linewidth=1.0):
    ...

plot(x, y, color="red")
```

The bare `*` in the signature means "everything after this is keyword-only." scikit-learn uses this pattern everywhere; it makes `KNeighborsClassifier(n_neighbors=5, weights="distance")` read like English.

3. `*args` and `**kwargs` capture variable-length positional and keyword arguments:

```
def log(level, *messages, **context):
    print(level, *messages, context)

log("INFO", "hello", "world", user="alice")
# INFO hello world {'user': 'alice'}
```

4. **Type hints** are documentation. They are not enforced at runtime. Tools like `mypy` and `pyright` check them statically. Use them for any function whose types aren't instantly obvious; skip them for throwaway scripts.

3.5 Files, paths, and reading data

Before NumPy and pandas, you sometimes read a file by hand. The idiomatic way, in plain Python:

```
from pathlib import Path

path = Path("data/small.csv")
lines = path.read_text().splitlines()

# Or, if the file is large and you want to stream:
with open(path, encoding="utf-8") as f:
    for line in f:
        ...
```

Use `pathlib.Path` rather than raw strings. It handles separators correctly on Windows, composes with `/`, and has convenience methods like `.stem`, `.suffix`, `.exists`.

```
>>> p = Path("data/sales/2024-01.csv")
>>> p.parent
PosixPath('data/sales')
>>> p.stem
'2024-01'
>>> p.suffix
'.csv'
>>> p.with_suffix(".parquet")
PosixPath('data/sales/2024-01.parquet')
```

For real data loading, however, you will almost always use pandas:

```
import pandas as pd
df = pd.read_csv("data/sales.csv")
```

That's it. Chapter 7 covers the options.

3.6 Errors and exceptions

Python signals failures by raising exceptions:

```
>>> int("banana")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'banana'
```

Catch one only when you have a specific recovery in mind:

```
def parse_or_none(s: str) -> int | None:
    try:
        return int(s)
    except ValueError:
        return None
```

Warning

Do not write `except:` or `except Exception:` without a reason. Both swallow errors you probably wanted to see, including the `KeyboardInterrupt` that stops a runaway cell. Always name the specific exception class you expect.

3.7 Classes, only as far as you need

You will read classes more often than you write them in data work. Most of your code is functions. But you need to know the shape:

```
class MovingAverage:
    """Streaming moving average over the last n values."""

    def __init__(self, n: int) -> None:
        self.n = n
        self.window: list[float] = []

    def add(self, x: float) -> float:
        self.window.append(x)
        if len(self.window) > self.n:
            self.window.pop(0)
        return sum(self.window) / len(self.window)

>>> ma = MovingAverage(3)
>>> [ma.add(x) for x in [1, 2, 3, 4, 5]]
[1.0, 1.5, 2.0, 3.0, 4.0]
```

Five things, in order of importance:

1. `__init__` is the constructor; `self` is the instance.
2. You never call `MovingAverage.__init__(x)` directly — calling the class, `MovingAverage(3)`, does that for you.
3. Instance attributes (`self.n` , `self.window`) are just ordinary assignments.
4. Methods are functions defined in the class body with `self` as the first parameter.
5. Dunder methods (`__len__` , `__add__` , `__iter__` , `__repr__`) integrate your class with the language. You won't need them often for data work; scikit-learn and pandas handle the integration for you.

If you find yourself writing something that's mostly data and no behavior, use a `dataclass` :

```
from dataclasses import dataclass

@dataclass
class ModelConfig:
    n_estimators: int = 100
    max_depth: int | None = None
    random_state: int = 0

>>> cfg = ModelConfig(n_estimators=200)
>>> cfg
ModelConfig(n_estimators=200, max_depth=None, random_state=0)
```

One line of `@dataclass` and you get a constructor, a readable `repr`, and equality comparison for free.

3.8 A data-cleaning warm-up

To close the chapter, here is a cleaning task done in pure Python. The point is to build a baseline that later chapters make dramatically shorter. Starting data: a comma-separated string with two numeric columns and a row of header.

```
raw = """height_cm,weight_kg
178.0,72.5
,68.0
165,55.2
171.3, 81
,
182.2,90.1"""

def parse_rows(text: str) -> list[dict]:
    lines = text.strip().splitlines()
    header = [h.strip() for h in lines[0].split(",")]
    rows = []
    for line in lines[1:]:
        cells = [c.strip() for c in line.split(",")]
        row = {}
        for key, cell in zip(header, cells):
            row[key] = float(cell) if cell else None
        rows.append(row)
    return rows

def dropna(rows: list[dict]) -> list[dict]:
    return [r for r in rows if all(v is not None for v in r.values())]

def mean(rows: list[dict], key: str) -> float:
    vals = [r[key] for r in rows]
    return sum(vals) / len(vals)

rows = dropna(parse_rows(raw))
print(f"n = {len(rows)}")
print(f"mean height = {mean(rows, 'height_cm'):.1f}")
print(f"mean weight = {mean(rows, 'weight_kg'):.1f}")
```



```
n = 4
mean height = 174.1
mean weight = 74.7
```

This is 20 lines of code to parse a four-row CSV, drop rows with missing values, and take two column means. In Chapter 7, the same task is:

```
import pandas as pd, io
df = pd.read_csv(io.StringIO(raw)).dropna()
print(f"n = {len(df)}")
print(df.mean().round(1).to_string())
```

Four lines. This is what we mean by "pandas does the boring parts."

3.9 Exercises

1. Write a function that takes a list of numbers and returns a tuple (mean, standard_deviation) . Use only the standard library.
2. Convert the MovingAverage class from Section 3.7 to use collections.deque(maxlen=n) instead of a list. Read the docs for deque if you haven't. What changes?
3. The cleaning code in Section 3.8 silently drops rows where any value is missing. Modify dropna to accept an optional keys argument that specifies which columns must be non-missing. Rows may have extra missing values in other columns.
4. Why does this print [1, 2] and not [] ?

```
def f(xs=[]):
    xs.append(len(xs) + 1)
    return xs

print(f())
print(f())
```

5. A list comprehension with a filter is sometimes clearer as a generator with filter and map . Rewrite [x * 2 for x in data if x > 0] using filter and map . Which do you prefer? Why?

From here on, the Python language stops being the focus. Part II begins with NumPy, where the arrays live.

CHAPTER

NumPy Arrays

Every number in every plot in every notebook in the scientific Python stack lives, eventually, in a NumPy `ndarray`. Pandas `Series` wrap one. Matplotlib's `plot` copies your data into one. Scikit-learn accepts either `DataFrame`s or `ndarray`s and converts the former to the latter. This is the library to learn first because it is the library underneath all the others.

This chapter covers the `ndarray` type — what it is, how to make one, how to slice it, and why operations on it are fast. The next chapter covers the thing that makes it *interesting*: broadcasting.

4.1 What an `ndarray` is

An `ndarray` is a contiguous block of bytes plus metadata that describes how to interpret them as a multidimensional array of uniform type. Three pieces of metadata matter most:

- `dtype` — the type of each element (`float64` , `int32` , `bool` , `uint8` , ...). Every element has the same type.
- `shape` — a tuple describing the size of each axis. `(3, 4)` is three rows by four columns.
- `strides` — how many bytes to skip to advance by one along each axis. This is how NumPy does slicing without copying.

```
import numpy as np

x = np.arange(12).reshape(3, 4)
print(x)
print(f"shape={x.shape}, dtype={x.dtype}, strides={x.strides}")
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
shape=(3, 4), dtype=int64, strides=(32, 8)
```

`strides=(32, 8)` means "move 32 bytes to go down one row, 8 bytes to go right one column." 8 bytes is the size of an `int64`; 32 is four of them, which makes sense because each row has four elements.

This is the key insight: **NumPy stores its data in C, not in Python.** A Python `list` of 100 integers holds 100 `PyObject*` pointers, each pointing to a boxed integer scattered across the heap. A NumPy `ndarray` of 100 integers holds $8 \cdot 100$ contiguous bytes. The speed difference is not subtle.

Note

`shape` is almost always a tuple. `x.shape = (100,)` is a 1-D array of 100 elements. `x.shape = (100, 1)` is a 2-D array, a column. These are different and NumPy will occasionally insist on the distinction.

4.2 Creating arrays

Five creation functions do 95% of the work:

```
np.array([1, 2, 3])           # from a Python sequence
np.zeros((3, 4))             # all zeros, shape (3, 4), float64
np.ones((2, 2), dtype=int)   # all ones, explicit dtype
np.arange(0, 10, 2)          # [0, 2, 4, 6, 8] — like range()
np.linspace(0, 1, 5)         # [0, 0.25, 0.5, 0.75, 1]
```

```
array([0, 2, 4, 6, 8])
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

Three more you will see regularly:

```
np.eye(3)                   # identity matrix
np.full((2, 3), -1.0)       # all -1.0
np.empty((2, 3))            # uninitialized memory — contents are garbage
```

`np.empty` exists because allocation is cheaper than initialization and sometimes you're about to overwrite every element anyway. For normal code, prefer `zeros` — an uninitialized array is a bug waiting to happen.

From random

`np.random.default_rng(seed)` returns a new-style random generator. It is the function to reach for; older `np.random.rand`-style calls are legacy and share global state.

```
rng = np.random.default_rng(seed=0)
rng.standard_normal((3, 4))           # shape (3,4), N(0,1)
rng.uniform(-1, 1, size=5)            # shape (5, ), U(-1,1)
rng.integers(0, 10, size=(2, 2))      # shape (2,2), ints in [0,10)
```

Seeding is non-negotiable for anything you want to reproduce. Fix the seed at the top of every script that uses randomness.

4.3 dtypes

NumPy's dtypes are more specific than Python's because they map to C types. The ones worth knowing:

dtype

C type

bytes

range / notes

`bool`

`char`

1

`True` / `False`

`int32`

`int32_t`

4

`±2.1 × 1016`

`int64``int64_t`

8

 $\pm 9.2 \times 10^{15}$, default on 64-bit`uint8``uint8_t`

1

0–255, image pixels

`float32``float`

4

single precision; GPU default

`float64``double`

8

double precision; numpy default

`complex128`two `double` s

16

FFT, signal processing

Type coercion follows "widening wins" rules:

```
>>> np.array([1, 2, 3]) + np.array([1.0, 2.0, 3.0])
array([2., 4., 6.])           # int + float -> float
>>> np.array([1], dtype=np.int32) + np.array([1], dtype=np.int64)
array([2])                   # int32 + int64 -> int64
```

Warning

`int32` overflow is silent in NumPy. `np.int32(2_000_000_000) * 2` gives you `-294967296` with no warning. When your numbers get large, cast up to `int64` or `float64` yourself.

4.4 Indexing and slicing

Slicing syntax is the same as Python lists — `start:stop:step` — but generalized to every axis with commas:

```
x = np.arange(12).reshape(3, 4)
x[0]           # row 0:           array([0, 1, 2, 3])
x[0, 1]        # element (0,1):  1
x[:, 1]        # column 1:       array([1, 5, 9])
x[1:3, 1:3]    # 2x2 submatrix:  array([[5, 6], [9, 10]])
x[::-1]       # rows reversed:  array([[8, 9, 10, 11], [4, 5, 6, 7], [0, 1, 2, 3]])
```

Two features that distinguish NumPy from lists:

Boolean indexing

A boolean array of the same shape selects the elements where it's true:

```
x = np.arange(10)
mask = (x % 2 == 0) & (x > 2)
x[mask]           # array([4, 6, 8])
```

Two gotchas:

1. Use `&`, `|`, `~` — **not** `and`, `or`, `not`. The Python keywords operate on whole arrays (which are truthy iff non-empty) and produce the wrong answer.
2. Parenthesize each comparison: `(x > 2) & (x < 8)`, because `&` binds tighter than `>`.

Fancy indexing

An integer array selects elements at those positions:

```
rows = np.array([0, 2])
x = np.arange(12).reshape(3, 4)
x[rows]
# array([[ 0,  1,  2,  3],
#        [ 8,  9, 10, 11]])
```

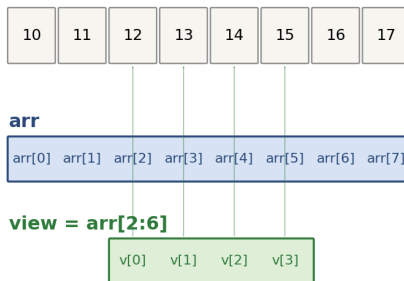
You can combine axes:

```
x[[0, 1, 2], [0, 1, 2]] # the diagonal: array([0, 5, 10])
```

4.5 Views vs copies

This is the single concept that trips up more NumPy users than any other. **Slicing returns a view; fancy indexing returns a copy.**

buffer (contiguous memory)



View: same memory, different index mapping

Modifying view[0] mutates arr[2] and the buffer

copy = arr[2:6].copy() → new buffer, no aliasing

Rule: slices are views; fancy indexing is a copy.

Slicing creates a view — a different shape and stride on the same buffer. Fancy indexing has to allocate a new array because the selected elements aren't a regular stride.

A view shares memory with the original. Writing to it changes the original:

```
x = np.arange(12).reshape(3, 4)
v = x[:, 1:3] # view
v[0, 0] = 99
print(x)
```

```
[[ 0 99  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Fancy indexing does *not* share memory:

```
f = x[[0, 2]]          # copy
f[0, 0] = 77
print(x)               # unchanged
```

```
[[ 0 99  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

You can check with `.base` : `v.base is x` is `True` ; `f.base is None` .

Tip

If you need a standalone copy from a slice, call `.copy()` explicitly: `v = x[:, 1:3].copy()` . It makes your intent obvious and prevents spooky action-at-a-distance later.

4.6 Reshaping and transposing

`reshape` changes how NumPy interprets the buffer without moving any bytes — the new shape must have the same total number of elements:

```
a = np.arange(12)
a.reshape(3, 4)          # 3 rows x 4 cols
a.reshape(2, 2, 3)       # 3-D: 2 x 2 x 3
a.reshape(-1, 4)         # -1 means "whatever works"
```

`-1` is a wildcard — there can be only one of them per call, and NumPy solves for its value. `a.reshape(-1, 4)` is "4 columns, whatever rows."

`x.T` transposes — also without copying, usually, since it's just a swap of shape and strides:


```
x = np.arange(12).reshape(3, 4)
x.T.shape           # (4, 3)
x.T.strides         # (8, 32) – flipped from x's (32, 8)
```

The transpose is a view. If you then call an operation that requires contiguous memory, NumPy may need to copy. The difference shows up in performance, not correctness.

4.7 Aggregations

Every array has `.sum()`, `.mean()`, `.min()`, `.max()`, `.std()`, `.var()`, `.argmin()`, `.argmax()`. All of them take an `axis` argument:

```
x = np.arange(12).reshape(3, 4)
x.sum()           # 66, the total
x.sum(axis=0)     # array([12, 15, 18, 21]) – column sums
x.sum(axis=1)     # array([ 6, 22, 38]) – row sums
```

Reading `axis=N` correctly: **"collapse axis N."** `axis=0` collapses the rows, so you get one number per column. `axis=1` collapses the columns, so you get one number per row.

```
x.mean(axis=1)    # array([1.5, 5.5, 9.5])
```

The result has one fewer dimension than the input, unless you pass `keepdims=True`:

```
x.mean(axis=1, keepdims=True)
# array([[1.5],
#        [5.5],
#        [9.5]])
```

`keepdims=True` is handy for broadcasting — see Chapter 5.

4.8 Saving and loading

NumPy has its own binary format:

```
np.save("arr.npy", x)
y = np.load("arr.npy")
```

`.npy` is a pickle-free, self-describing format that preserves dtype and shape. `.npz` (via `np.savez` / `np.savez_compressed`) bundles multiple arrays into one file.

For text files (CSVs of numbers, no headers, no strings), `np.loadtxt` and `np.savetxt` work but they are slow for anything real. Use pandas for text data — we cover that in Chapter 7.

4.9 Exercises

1. Create a 10x10 array of random integers between 0 and 99 with a fixed seed. Extract the main diagonal and the anti-diagonal. Verify that they have length 10.
2. Given `x = np.arange(20).reshape(4, 5)`, without using a loop, produce: (a) every element that is a multiple of 3; (b) every element in an even-indexed row; (c) every element whose row index equals its column index (the diagonal).
3. Explain in one sentence each why the following pairs differ: (a) `x[:, 0]` and `x[:, 0:1]`; (b) `x[[0, 1]]` and `x[0:2]` (what do they share, what do they not); (c) `x.sum()` and `x.sum(axis=0).sum()`.
4. Write a function `standardize(x)` that takes a 2-D array and returns a new array where every column has mean 0 and standard deviation 1. No loops.
5. Using `%timeit` in Jupyter (or `timeit` from the command line), compare the cost of summing `list(range(10**6))` with Python's `sum` versus summing `np.arange(10**6)` with `.sum()`. Roughly how many times faster is NumPy?

Chapter 5 is about what makes NumPy a language rather than a library: broadcasting, the rule that lets `a + b` work for almost any pair of compatibly-shaped arrays.

CHAPTER

Vectorization and Broadcasting

A Python `for` loop over ten million numbers is slow because every iteration pays the cost of the Python interpreter: a bytecode dispatch, a type check, an allocation. A NumPy `+` on two arrays of ten million numbers is fast because the loop is compiled C and the elements are a contiguous block of doubles. The programmer's job is to express whatever they wanted to do as array operations that let that C loop run.

That skill is called *vectorization*. The tool that makes it possible for arrays of different shapes is called *broadcasting*. This chapter covers both.

5.1 Why loops are slow, visibly

Here is a speed comparison you can reproduce. Summing 10⁷ numbers:

```
import numpy as np
from time import perf_counter

n = 1_000_000
xs = list(range(n))

t0 = perf_counter(); total = 0
for x in xs: total += x
t_list = perf_counter() - t0

arr = np.arange(n)
t0 = perf_counter(); total = arr.sum()
t_np = perf_counter() - t0

print(f"python list: {t_list*1000:.1f} ms")
print(f"numpy sum:   {t_np*1000:.2f} ms")
print(f"speedup:     {t_list / t_np:.0f}x")
```

A representative run on my laptop:

```
python list: 35.4 ms
numpy sum:   0.42 ms
speedup:     84x
```

A factor of ~ 100 is typical for simple reductions. For more involved operations — element-wise math followed by a reduction — the speedup can be 1000 $\times$ or more, because NumPy avoids building Python objects entirely.

The moral: when performance matters, your loop over values should turn into a single expression on arrays. That's it. That's the whole idea.

5.2 Universal functions

A *universal function* (ufunc) is a NumPy function that operates element-wise on arrays. All the arithmetic operators are ufuncs:

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([10, 20, 30])
x + y          # array([11., 22., 33.])
x * y          # array([10., 40., 90.])
np.sqrt(x)     # array([1.      , 1.4142, 1.7320])
np.exp(y)      # array([2.2026e+04, 4.8517e+08, 1.0686e+13])
np.sin(x)      # array([0.8414, 0.9092, 0.1411])
```

Crucially, you can mix scalars and arrays:

```
x * 2          # array([2., 4., 6.])    – scalar "broadcasts"
x - x.mean()   # centering: no loop needed
```

And you can chain them:

```
z = np.sin(x) ** 2 + np.cos(x) ** 2
# array([1., 1., 1.])
```

Each arithmetic expression is one C loop. Chaining four operations produces four passes over the array, which is still orders of magnitude faster than a Python loop — but if you are doing dozens of operations you may want a compiler like `numba` or `jax` that can fuse them.

5.3 Broadcasting

Broadcasting is the rule that lets NumPy operate on arrays whose shapes don't quite match by "stretching" the smaller one without actually copying.

a, shape (3,)

1	2	3
---	---	---

a + b, shape (4, 3) — broadcast

Align shapes from the RIGHT.
Broadcasting rule
 For each dim, sizes must either match
 or one of them must be 1 (then it is
 stretched to match the other).

b, shape (4,1)

10
20
30
40

11	12	13
21	22	23
31	32	33
41	42	43

Shapes here:

a: (3,) → (1, 3)

b: (4, 1)

→ broadcast to (4, 3)

No data is copied: NumPy uses a
 stride of 0 on the broadcast axis.

Broadcasting `a` (shape `(3,)`) with `b` (shape `(4,1)`) gives a `(4,3)` array. Each axis of size 1 (or implicitly 1, from padding on the left) is stretched to the size of the other operand. No data is copied — NumPy uses a stride of zero on the broadcast axis.

The rule in one line

Align shapes from the right. For each aligned axis, sizes must match or one of them must be 1. A size-1 axis is stretched.

Worked examples:

```
a = np.array([1, 2, 3])           # shape (3,)
b = np.array([[10], [20], [30], [40]]) # shape (4, 1)

a + b
# array([[11, 12, 13],
#        [21, 22, 23],
#        [31, 32, 33],
#        [41, 42, 43]])
```

How that gets aligned: pad `a` on the left with implicit 1s to match `b`'s number of axes: `(1, 3)` vs `(4, 1)`. Now each axis either matches or one of them is 1. Stretch both. Result: `(4, 3)`.

A 1-D example that catches everyone at least once:

```
x = np.array([1, 2, 3])           # shape (3,)
y = x[:, None]                   # shape (3, 1)
x + y
# array([[2, 3, 4],
#        [3, 4, 5],
#        [4, 5, 6]])
```

`[:, None]` is the idiomatic way to add an axis. `np.newaxis` is a synonym.

Shapes that don't broadcast

```
>>> np.zeros(3) + np.zeros(4)
Traceback (most recent call last):
...
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

Aligned from the right: `(3,)` vs `(4,)` — neither matches, neither is 1. Failure. NumPy's error messages at least tell you exactly what shapes it tried to align.

Tip

When broadcasting confuses you, print `.shape` for both operands, add implicit leading 1s until they have the same number of axes, and check axis by axis. Nine times out of ten the bug is a missing `[:, None]`.

5.4 Vectorizing common idioms

Here are the operations you will reach for again and again.

Centering a dataset

```
X = rng.standard_normal((100, 4))  # 100 samples, 4 features
X_centered = X - X.mean(axis=0)     # subtract per-column mean
```

`X.mean(axis=0)` has shape `(4,)`. Broadcasting pads it to `(1, 4)` and stretches the first axis to 100. One C loop over the whole array.

Standardizing

```
X_std = (X - X.mean(axis=0)) / X.std(axis=0)
print(X_std.mean(axis=0).round(2))    # [ 0.  0.  0.  0.]
print(X_std.std(axis=0).round(2))     # [ 1.  1.  1.  1.]
```

Pairwise Euclidean distances

A loop-free way to compute the (n, n) matrix of distances between every pair of points in an (n, d) array:

```
def pairwise_distances(X):
    # |a - b|^2 = |a|^2 + |b|^2 - 2 a·b
    sq = (X ** 2).sum(axis=1)          # (n,)
    d2 = sq[:, None] + sq[None, :] - 2 * X @ X.T
    np.maximum(d2, 0, out=d2)          # guard against tiny negatives
    return np.sqrt(d2)

pts = rng.standard_normal((5, 3))
D = pairwise_distances(pts)
D.shape                               # (5, 5)
np.diag(D)                           # array([0., 0., 0., 0., 0.])
```

This is a standard trick — expanding a squared distance using the dot product — that you will see again in Chapters 14 and 16.

One-hot encoding

```
labels = np.array([0, 2, 1, 0, 2])    # integer labels, 3 classes
n_classes = 3
onehot = np.eye(n_classes)[labels]
# array([[1., 0., 0.],
#        [0., 0., 1.],
#        [0., 1., 0.],
#        [1., 0., 0.],
#        [0., 0., 1.]])
```

`np.eye(n_classes)[labels]` is a fancy-indexing classic: each row is a one-hot row of the identity matrix, and `labels` picks them out.

Masking with `np.where`

`np.where(condition, a, b)` returns `a` where the condition is true, `b` where false — element-wise:

```
x = rng.standard_normal(8)
x_pos = np.where(x > 0, x, 0)          # ReLU
```

For more than two branches, `np.select` does the same thing with a list of conditions and values.

5.5 Reshaping for broadcasting

Adding and removing size-1 axes is the day-to-day work of making shapes line up.

```
a = np.arange(6)                # shape (6,)
a.reshape(2, 3)                 # (2, 3)
a.reshape(-1, 1)               # (6, 1) — column vector
a[:, None]                     # same thing, shorter
a[None, :]                     # (1, 6) — row vector
a.reshape(1, 2, 3, 1)          # (1, 2, 3, 1)
np.squeeze(a.reshape(1, 2, 3, 1)) # (2, 3) — drops size-1 axes
np.expand_dims(a, axis=0)       # (1, 6)
```

Two short idioms I use constantly:

```
# Per-row softmax: subtract max for numerical stability, exp, normalize.
scores = rng.standard_normal((5, 3))
s = scores - scores.max(axis=1, keepdims=True)
probs = np.exp(s)
probs /= probs.sum(axis=1, keepdims=True)
probs.sum(axis=1)                # array([1., 1., 1., 1., 1.])
```

`keepdims=True` is what makes the subsequent broadcast work: without it, `max` returns shape `(5,)` which would align with the last axis, not the first.

5.6 When vectorization is hard

Not every algorithm vectorizes cleanly. The honest answer in those cases is usually one of:

1. **Write the loop anyway.** If it's called once, on a small input, the readability win outweighs the speed loss.
2. **Use `numba`.** Decorate a numeric function with `@numba.njit` and it gets compiled to machine code. You write loops, you get C speed.
3. **Use `np.vectorize`.** A convenience wrapper that makes a scalar function look array-aware. It does not actually run faster than a Python loop; it's just a way to get broadcasting-friendly APIs.

In practice, 80% of data code vectorizes trivially, 15% vectorizes with some thought, and 5% is genuinely easier as a loop. Don't fight the 5%.

5.7 Exercises

1. Given an `(n, 2)` array of 2-D points, compute the pairwise Euclidean distance matrix of shape `(n, n)` without a loop. Check your answer against `scipy.spatial.distance.pdist` for `n = 20`.
2. Build a 10×10 multiplication table (no loops). Hint: use `np.arange` twice and broadcasting.
3. Given an `(n, d)` array `X`, compute the `(d, d)` covariance matrix using only NumPy. Compare with `np.cov(X, rowvar=False)`. Pay attention to whether your denominator is `n` or `n - 1`.
4. Given `x = np.linspace(0, 1, 11)`, compute a second array `y` where `y[i] = i * x[i]` without using an explicit loop or `enumerate`.
5. Read the docs for `np.einsum`. Express the matrix multiply `A @ B` and the pairwise-distance squared-norm `(a ** 2).sum(axis=1)` using it. Einsum pays off for any computation with more than two tensor indices.

Chapter 6 — the last of the NumPy three — is linear algebra and randomness: the parts of the library you reach for when you move from data manipulation to actually computing things.

Linear Algebra and Random Numbers

NumPy is a linear-algebra library with a data-manipulation layer on top. This chapter covers both: the matrix operations that underlie essentially every model in this book, and the random-number tools you use to simulate, sample, and shuffle. It's short because the API is compact — but the ideas in it show up everywhere from Chapter 13 onward.

6.1 Matrix multiplication and `@`

NumPy distinguishes **element-wise** multiplication (`*`) from **matrix** multiplication (`@` or `np.matmul`). Mixing them up is the most common bug in a notebook that is mostly working.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A * B           # element-wise
# array([[ 5, 12],
#        [21, 32]])

A @ B           # matrix multiply
# array([[19, 22],
#        [43, 50]])
```

`@` has been a real operator in Python since 3.5 and reads much better than `np.matmul(A, B)`. For 1-D arrays it's the dot product:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
a @ b           # 1*4 + 2*5 + 3*6 = 32
```

For a mix of 2-D and 1-D it does what you'd expect:

```
A @ a           # treats a as a column, returns 1-D
# array([14, 32])
```

For higher-dimensional arrays `@` broadcasts over all but the last two dimensions, which is how batched matrix multiplies work in PyTorch, JAX, and NumPy alike.

6.2 Solving linear systems

To solve $Ax = b$ for x , use `np.linalg.solve` — not `np.linalg.inv(A) @ b`. Inversion is numerically worse and usually slower.

```
A = np.array([[2., 1.],
              [1., 3.]])
b = np.array([5., 10.])

x = np.linalg.solve(A, b)
print(x)                # [1. 3.]
print(A @ x)            # [ 5. 10.] — check
```

`solve` uses LU decomposition with partial pivoting: $O(n^3)$ for $n \times n$, which is fine for the sizes you'll see in data work.

6.3 The other decompositions

`np.linalg` has a handful of factorizations. Three matter:

Eigendecomposition

For a real square matrix A , find scalars λ and vectors v such that $Av = \lambda v$:

```
eigvals, eigvecs = np.linalg.eig(A)
print(eigvals)      # [1.382 3.618]
```

Columns of `eigvecs` are the eigenvectors. Use `np.linalg.eigh` if A is symmetric — it is faster and guaranteed to return real eigenvalues.

Applications you will see later: PCA (the eigenvectors of the covariance matrix), spectral clustering, power iteration.

SVD

The singular-value decomposition factors any $m \times n$ matrix $A = U \Sigma V^T$, where U is $m \times m$ orthogonal, Σ is $m \times n$ diagonal with non-negative values, and V is $n \times n$ orthogonal.

```
U, S, Vt = np.linalg.svd(A, full_matrices=False)
print(S)           # singular values, descending
```

SVD underlies low-rank approximation (truncate to the top- k singular values), the pseudo-inverse, PCA again, and every recommendation system built in the 2000s.

Pseudo-inverse and least squares

For the overdetermined system $Ax \approx b$ (more rows than columns), the least-squares solution is $x = A^\dagger b$, where $A^\dagger$ is the Moore–Penrose pseudo-inverse. `np.linalg.lstsq` computes this directly:

```
# Linear regression  $y = X\beta$  (with intercept column already in  $X$ )
rng = np.random.default_rng(0)
n, d = 100, 3
X = rng.standard_normal((n, d))
beta_true = np.array([1.0, -2.0, 0.5])
y = X @ beta_true + 0.1 * rng.standard_normal(n)

beta_hat, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print(beta_hat.round(3))
```

```
[ 0.999 -2.006  0.509]
```

This is a complete linear regression in four lines of NumPy. Chapter 15 repeats it with `scikit-learn`, but the math is what `lstsq` does under the hood.

6.4 Randomness done right

All the examples so far have used a *random generator* created by `np.random.default_rng(seed)`. This is the modern API; the older `np.random.seed(...) + np.random.rand(...)` style is global, thread-unsafe, and deprecated.

```
rng = np.random.default_rng(seed=42)
```

Methods on a generator correspond to distributions:

```
rng.standard_normal(5)          # N(0, 1)
rng.normal(loc=3.0, scale=2.0, size=10)
rng.uniform(low=-1, high=1, size=(3, 3))
rng.integers(low=0, high=10, size=5) # [0, 10) – half-open
rng.choice([10, 20, 30], size=4, replace=True, p=[0.2, 0.3, 0.5])
rng.permutation(np.arange(5))
```

For reproducibility, pass explicit seeds and never rely on `time.time()` implicitly.

Independent streams

When you write parallel code, you want different workers to have independent random streams that, together, reproduce deterministically. The right pattern is to spawn:

```
rng = np.random.default_rng(42)
child_rngs = rng.spawn(4)          # 4 independent generators

for i, r in enumerate(child_rngs):
    sample = r.standard_normal(3)
    print(f"worker {i}: {sample.round(3)}")
```

```
worker 0: [-1.176  0.229 -1.128]
worker 1: [-1.101  0.523 -0.608]
worker 2: [ 1.182  0.168  0.138]
worker 3: [-0.082 -0.166  0.338]
```

6.5 Monte Carlo: putting it together

A good way to combine everything in this chapter is to estimate π by throwing darts at a unit circle inscribed in a square. The ratio of points that land inside the circle to the total is $\pi/4$:

```

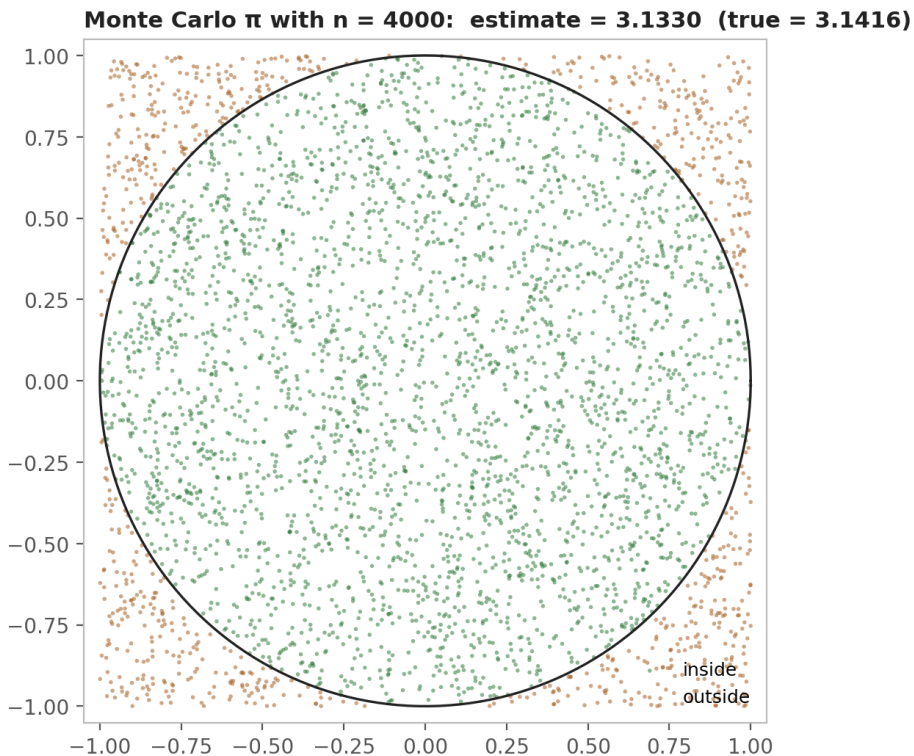
rng = np.random.default_rng(0)
n = 4000
pts = rng.uniform(-1, 1, size=(n, 2))
inside = (pts ** 2).sum(axis=1) <= 1.0
pi_est = 4.0 * inside.mean()
print(f"pi ≈ {pi_est:.4f}    (n = {n})")

```

```

pi ≈ 3.1230    (n = 4000)

```



Monte Carlo estimate of π with 4,000 uniformly distributed points in the square $[-1, 1]^2$. Green points fall inside the unit circle; orange points fall outside. The estimate $4 \times |\text{inside}| / n$ converges to π at a rate of $O(1/\sqrt{n})$.

Four lines for the core computation, and every line is a vectorized expression over a 4,000-element array. No loops; no intermediate Python objects. Scale `n` up to 10⁶ and the same code runs in under a second.

6.6 Exercises

1. Build a 3×3 rotation matrix $R(\theta)$ for angle θ radians. Show that for $\theta = \pi/6$, applying R three times rotates a vector by $\pi/2$. Use floating-point tolerance, not `==`.
2. Given a tall skinny matrix X (e.g. 1000×5) with independent standard-normal columns, compute its singular values. What do you expect them to be, approximately? Why? (Hint: Marchenko–Pastur.)
3. Implement linear regression from scratch: (a) generate synthetic data $y = X\beta + \epsilon$; (b) solve for β with `np.linalg.lstsq`; (c) verify that β is close to β ; (d) compute residuals and their variance.
4. Estimate π via Monte Carlo for $n = 10^{**k}$ with $k = 2, 3, 4, 5, 6$. Plot the absolute error vs. n on a log–log scale. What slope do you get?
5. `np.linalg.inv(A) @ b` and `np.linalg.solve(A, b)` return the same answer in exact arithmetic. Construct an A where they differ noticeably in floating point. Hint: nearly singular matrices.

Part II is done. You now have all the NumPy you need to understand what pandas does on top of it. Part III begins with the `DataFrame` — a dictionary of NumPy arrays with a shared index, and the bulk of what a working analyst uses day to day.

CHAPTER

Series and DataFrame

Pandas is two data structures and a lot of methods on them. The two structures are `Series` (a 1-D labelled array) and `DataFrame` (a 2-D table that is a dict of `Series` sharing an index). This chapter introduces both, how they're laid out, and how to get data in and out.

The rest of Part III is about the things you do *with* a `DataFrame`. This chapter is about the `DataFrame` itself.

7.1 The Series

A `Series` is a NumPy array with a label on every element. That's it.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40],
              index=["a", "b", "c", "d"],
              name="counts")

print(s)
```

```
a    10
b    20
c    30
d    40
Name: counts, dtype: int64
```

Three things to see:

1. **The index.** `["a", "b", "c", "d"]` is the index. It provides a label for each element. Indices don't have to be unique, but almost always should be.
2. **The dtype.** Same rules as NumPy — every element has the same type, and the Series prints it at the bottom.
3. **The name.** Optional, but every `Series` has a `.name`. When you pull one out of a DataFrame, its name is the column name.

You can pull the underlying array out when you need to:

```
s.values      # numpy array: array([10, 20, 30, 40])
s.index       # Index(['a', 'b', 'c', 'd'], dtype='object', name=None)
s.to_numpy()  # same as .values but preferred in new code
```

7.2 The DataFrame

A `DataFrame` is a dict of `Series` that share an index:


```
df = pd.DataFrame({
    "city": ["Seoul", "Tokyo", "Osaka", "Busan", "Kyoto"],
    "temp_c": [-3.2, 4.1, 3.5, 0.9, 2.8],
    "humid": [55, 61, 58, 63, 59],
}, index=pd.to_datetime(["2024-01-01", "2024-01-02", "2024-01-03",
                        "2024-01-04", "2024-01-05"]))
df.index.name = "date"
print(df)
```

	city	temp_c	humid
date			
2024-01-01	Seoul	-3.2	55
2024-01-02	Tokyo	4.1	61
2024-01-03	Osaka	3.5	58
2024-01-04	Busan	0.9	63
2024-01-05	Kyoto	2.8	59

Anatomy of a DataFrame

index	city	temp_c	humid
2024-01-01	Seoul	-3.2	55
2024-01-02	Tokyo	4.1	61
2024-01-03	Osaka	3.5	58
2024-01-04	Busan	0.9	63

← columns (Index)

Each column is a Series — a 1-D labelled NumPy array, sharing the DataFrame's index.

dtypes (per column):

```
city    object (Python strings)
temp_c  float64
humid   int64
```

The index is not a column. You select it with `.index` and label-index with `.loc['2024-01-02']`.

A DataFrame has an index (row labels), a columns Index (column labels), and per-column dtypes. Each column is a Series; they all share the same index. The index is not itself a column — you select rows by label with `.loc[...]`, not with `df["date"]`.

Every DataFrame has three things worth knowing in your first minute with it:

```
df.shape      # (5, 3) – rows x columns
df.dtypes     # per-column dtypes
df.columns    # Index(['city', 'temp_c', 'humid'], dtype='object')
df.index      # DatetimeIndex(['2024-01-01', ...], ...)
```

And two more:

```
df.info()     # shape, dtypes, memory, missing count
df.describe() # summary stats for numeric columns
```

`df.info()` is the first thing I run on any unfamiliar table. It tells you how many rows, what types, how much memory, and crucially where the missing values are.

7.3 Selecting a column

Three syntaxes do (nearly) the same thing:

```
df["city"]      # bracket – always works
df.city         # attribute – works if the column name is a valid identifier
df[["city", "humid"]] # list of names – returns a DataFrame
```

	city	humid
date		
2024-01-01	Seoul	55
2024-01-02	Tokyo	61
...		

The first is the one to reach for. The attribute form fails silently if the column name shadows a DataFrame method (e.g. a column named `shape`), so keep it for quick interactive use.

7.4 Selecting rows

Two kinds of selection, with two different tools:

- **By label** — `.loc`. The label is whatever your index contains: a string, a date, an integer.
- **By position** — `.iloc`. Zero-based integer indices, just like NumPy.

```
df.loc["2024-01-02"]
```

```
city      Tokyo
temp_c    4.1
humid     61
Name: 2024-01-02 00:00:00, dtype: object
```

```
df.iloc[1]
```

Same row, different way to get to it. When your index is a `DatetimeIndex`, label-based indexing lets you use natural slices:

```
df.loc["2024-01-02":"2024-01-04"] # inclusive of both endpoints
```

Warning

`.loc[a:b]` is inclusive of `b`. This is different from Python slicing (which is exclusive of the stop) and from `.iloc[a:b]` (which is also exclusive). It takes time to get used to. The rule is: label slices are closed; positional slices are half-open.

Boolean filtering works the same way as NumPy:

```
df[df["temp_c"] > 2]
```

	city	temp_c	humid
date			
2024-01-02	Tokyo	4.1	61
2024-01-03	Osaka	3.5	58
2024-01-05	Kyoto	2.8	59

For a row and a column at once, use `.loc[row, col]` or `.iloc[i, j]`:

```
df.loc["2024-01-03", "temp_c"] # 3.5
df.iloc[2, 1]                  # same
```

7.5 Reading and writing files

CSV

```
df = pd.read_csv("data/sales.csv")
df.to_csv("data/out.csv", index=False)
```

`read_csv` is the most featureful function in pandas. A handful of arguments you'll reach for:

```
pd.read_csv(
    "data/sales.csv",
    parse_dates=["order_date"],          # turn string -> datetime
    dtype={"zip": str},                  # keep leading zeros
    na_values=["-", "unknown", "N/A"],  # custom missing markers
    thousands=",",                      # 1,234 -> 1234
    index_col="order_id",
)
```

`index=False` when writing is usually what you want; otherwise pandas writes the DataFrame index as a first column, which is fine for pandas but not for other tools.

Excel

```
df = pd.read_excel("data.xlsx", sheet_name="Sales")
df.to_excel("out.xlsx", sheet_name="Sales", index=False)
```

Requires `openpyxl` (for `.xlsx`) or `xlrd` (for old `.xls`). Install one of them or you'll get a helpful error.

Parquet

For anything bigger than a few MB, use Parquet:

```
df.to_parquet("data.parquet")
df = pd.read_parquet("data.parquet")
```

Parquet is a columnar binary format. Reads and writes are 10–100× faster than CSV, dtypes round-trip correctly, and files are typically 5–10× smaller. The only reasons to prefer CSV are human-readability and compatibility with tools that don't speak Parquet (which is a shrinking list).

JSON

```
df.to_json("data.json", orient="records", lines=True)
df = pd.read_json("data.json", lines=True)
```

`orient="records"` plus `lines=True` produces one JSON object per line — the format everyone agrees on for stream-processable data.

7.6 Dtypes that aren't NumPy

Pandas has a few dtype families beyond NumPy's:

`object` — catch-all for strings and mixed types. Every Python string column is `object`. Slower than numeric dtypes; use `string[pyarrow]` (below) for large string columns.

`category` — labels stored as integer codes with a separate mapping. Saves memory and speeds up groupby on repeated string values:

```
df["city"] = df["city"].astype("category")
df["city"].memory_usage(deep=True)
```

`datetime64[ns]` — nanosecond timestamps. Produced by `pd.to_datetime`, `pd.date_range`, and `read_csv(parse_dates=...)`.

`Int64`, `Float64`, `boolean`, `string` (pandas-native nullable dtypes, capital letter) — support missing values without falling back to `NaN`:

```
s = pd.Series([1, 2, None], dtype="Int64")
# 0      1
# 1      2
# 2    <NA>
# dtype: Int64
```

`string[pyarrow]` — Arrow-backed string type, ~10× faster than `object` for string-heavy operations. Opt in per-column with `.astype("string[pyarrow]")` or flip the global default with `pd.options.future.infer_string = True`.

7.7 Memory

You can see how much memory a DataFrame uses:

```
df.memory_usage(deep=True)
```

`deep=True` is the important part — without it, object columns report only the size of their pointers, not the underlying strings. For a 1-million-row table of mostly strings, the difference can be two orders of magnitude.

If a table doesn't fit in memory at full precision, downcast:

```
# Safe downcasts when values allow
df["count"] = pd.to_numeric(df["count"], downcast="integer") # to int8/16/32 if it
df["price"] = pd.to_numeric(df["price"], downcast="float")   # to float32
df["city"] = df["city"].astype("category")                  # repeated strings
```

A DataFrame that started at 2 GB often fits in 400 MB after these steps, with no information loss.

7.8 A quick tour of what's next

```
df["temp_f"] = df["temp_c"] * 9 / 5 + 32 # assign a new column
df = df.rename(columns={"humid": "humidity"}) # rename
df = df.sort_values("temp_c", ascending=False) # sort
df.head(3); df.tail(3) # first/last rows
```

Chapter 8 is about selection, cleaning, and missing data. Chapter 9 is about groupby, joins, and time series. They cover most of what you do with DataFrames in a working analysis.

7.9 Exercises

1. Read the file `data/sales.csv` (10 rows, two columns: `order_date`, `amount`) such that dates are parsed as `datetime64` and `amount` is `float`. Confirm dtypes.
2. From the DataFrame in Section 7.2, produce: (a) rows where `temp_c < 0`; (b) rows where `humid >= 60`; (c) a single-row DataFrame for Tokyo.
3. Add a column `comfort` that is `"cold"` if `temp_c < 0`, `"mild"` if `0 <= temp_c < 10`, and `"warm"` otherwise. Use `np.select` or `pd.cut`. (`pd.cut` is cleaner when your cutoffs are numeric.)

4. Convert the `city` column to `category` . How much memory do you save on a 1,000-row version of the DataFrame where cities repeat?
5. Read the same file twice — once as CSV, once as Parquet (write it first). Compare read time with `%timeit` . How much faster is Parquet on your machine?

Chapter 8 is the other half of what it means to wield a DataFrame: selecting, filtering, cleaning, and the surprising amount of pandas that is about getting data *ready*.

CHAPTER

Selecting, Filtering, Cleaning

Most of the time spent "doing data work" is spent cleaning data. The tools are not glamorous and the techniques are not deep, but they compound: the analyst who is fast at the boring parts gets to spend more of their attention on the interesting parts. This chapter is about getting fast.

8.1 `loc` vs `iloc` , properly

We met `.loc` and `.iloc` in Chapter 7. Here is the complete story.

- `.loc[row_labels, col_labels]` selects by label.
- `.iloc[row_positions, col_positions]` selects by 0-based position.

Both accept:

- a single label/position → a scalar or a 1-D result
- a list of labels/positions → a subset
- a slice (labels are inclusive, positions are exclusive)
- a boolean array of the right length

```
# By label
df.loc["2024-01-02", "temp_c"]
df.loc[:, ["city", "humid"]]
df.loc["2024-01-02":"2024-01-04"]
df.loc[df["temp_c"] > 2, "city"]

# By position
df.iloc[1, 1]
df.iloc[:, [0, 2]]
df.iloc[0:3]
df.iloc[[0, 2, 4], :]
```

The one idiom that takes practice:

```
mask = df["temp_c"] > 2
df.loc[mask, "city"] = df.loc[mask, "city"].str.upper()
```

The mask is the rows; `"city"` is the column. This reads as "for the rows where `temp > 2`, uppercase the city name." It is the right way to conditionally assign — never do `df[mask]["city"] = ...`, which sets a value on a temporary view (see below).

8.2 The `SettingWithCopyWarning`

Pandas will sometimes return a *view* and sometimes a *copy*, in places where NumPy is predictable. The warning exists because this is unreliable:

```
subset = df[df["temp_c"] > 0]
subset["humid"] = 0 # SettingWithCopyWarning
```

The warning is pandas telling you "I don't know if `subset` is a view into `df` or its own copy, so I can't promise this assignment landed in the right place." The fix is always to do the assignment through `.loc`, on the original DataFrame:

```
df.loc[df["temp_c"] > 0, "humid"] = 0
```

Or, if you really meant to work on a copy:

```
subset = df[df["temp_c"] > 0].copy()
subset["humid"] = 0
```


Tip

If you see the warning, **stop** and rewrite the code through `.loc`. It's not "just a warning" — it means your assignment might have silently failed to change anything. In pandas 3.0, the warning becomes an error by default, so getting this right now is investment, not pedantry.

8.3 Missing data

Pandas uses `NaN` (from NumPy), `NaT` (for missing datetimes), and `None` (for object columns) to represent missingness. Newer nullable dtypes also introduce `<NA>`. For most work you can treat all of them as "missing."

Three methods you will use every day:

```
df.isna()           # boolean DataFrame, True where missing
df.dropna()         # drop any row with a missing value
df.fillna(0)        # replace missing with a value
```

`dropna` has useful arguments:

```
df.dropna(subset=["temp_c"])  # drop rows missing temp_c, ignore others
df.dropna(axis=1)            # drop columns that contain any NaN
df.dropna(how="all")         # drop rows where all values are NaN
```

`fillna` is similarly flexible:

```
df.fillna(df.mean(numeric_only=True))  # per-column mean
df["city"].fillna("unknown")           # string fill
df.fillna(method="ffill")               # carry the last known value forward
```

Forward-fill (`ffill`) is particularly useful for time series where a value is only reported on change.

For a summary of where the missing values actually are:

```
df.isna().sum().sort_values(ascending=False)
```

This is my first question about any new table: "how much is missing and where?"

8.4 Type coercion

Reading a CSV gives you strings for anything that doesn't look like a number. You'll convert columns constantly:

```
df["order_date"] = pd.to_datetime(df["order_date"])
df["amount"]     = pd.to_numeric(df["amount"], errors="coerce")
df["is_refund"]  = df["is_refund"].astype("boolean")
df["category"]   = df["category"].astype("category")
```

`errors="coerce"` turns un-parseable values into `NaN` instead of raising. Useful for dirty data; dangerous if you're expecting clean.

8.5 String methods

Every `Series` with string values has a `.str` accessor whose methods apply element-wise:

```
s = pd.Series([" Alice ", "BOB", "carol smith"])
s.str.strip()          # strip whitespace
s.str.lower()          # lowercase
s.str.split()          # split on whitespace -> list[str] per element
s.str.contains("li")   # boolean Series
s.str.replace("o", "0") # character replacement
s.str.extract(r"(\w+)\s+(\w+)") # regex groups -> DataFrame
```

```
0    alice
1     bob
2  carol smith
dtype: object
```

Combine with `.str.cat` to join, and you have 90% of the string transformation you'll ever need.

8.6 Datetime methods

For a column of datetimes, `.dt` is the analogue of `.str`:

```
df["order_date"].dt.year
df["order_date"].dt.month
df["order_date"].dt.dayofweek      # 0 = Monday
df["order_date"].dt.day_name()
df["order_date"].dt.to_period("M") # monthly period
```

```
(df["order_date"].max() - df["order_date"].min()).days # window span
```

The full set is documented under `pandas.Series.dt`. It overlaps with `numpy.datetime64` but adds per-element accessors that NumPy doesn't have.

8.7 `assign` and method chaining

`DataFrame.assign` returns a *new* `DataFrame` with additional columns computed from the existing ones. Combined with method chaining, it lets you express a transformation as a readable pipeline:

```
result = (
    df
    .dropna(subset=["amount"])
    .assign(
        month = lambda d: d["order_date"].dt.to_period("M"),
        amount_usd = lambda d: d["amount"] * 1.10,
    )
    .query("amount_usd > 50")
    .sort_values("amount_usd", ascending=False)
    .head(10)
)
```

The `lambda d: ...` inside `.assign` is important: each callback receives the `DataFrame` *after* previous steps, so later columns can depend on earlier transformations. Without the `lambda`, `d` would be the original `df` — and the `month` you just computed wouldn't be there yet.

Method chaining is a style choice. The benefit is that you read the pipeline top-to-bottom without intermediate names to keep track of. The cost is that you cannot `print` something in the middle without breaking the chain. `pd.DataFrame.pipe` helps when you want a debug hook:

```
def debug(d, label):
    print(label, d.shape); return d

(df.dropna().pipe(debug, "after dropna")
 .assign(x=lambda d: d["a"] + d["b"])
 .pipe(debug, "after assign"))
```

8.8 A small cleaning worked example

Put it all together. Here is a dirty CSV and a clean version of it, without editorialising:

```
import pandas as pd, io

raw = """order_id,order_date,amount,customer
1,2024-01-15,$120.50,alice
2,01/16/2024,45,bob
3,2024-01-17,, carol
4,2024-01-18,$1,200.00,ALICE
5,,,
6,2024-01-19,$75.00,bob
"""

df = (
    pd.read_csv(io.StringIO(raw))
    .dropna(subset=["order_date"])
    .assign(
        order_date = lambda d: pd.to_datetime(d["order_date"],
                                              format="mixed",
                                              errors="coerce"),
        amount      = lambda d: (d["amount"]
                                .astype(str)
                                .str.replace(r"[\$,]", "", regex=True)
                                .pipe(pd.to_numeric, errors="coerce")),
        customer    = lambda d: d["customer"].str.strip().str.lower(),
    )
    .dropna(subset=["amount"])
    .reset_index(drop=True)
)

print(df)
print()
print(df.dtypes)
```

```

    order_id order_date  amount customer
0         1 2024-01-15   120.5    alice
1         2 2024-01-16    45.0     bob
2         4 2024-01-18  1200.0    alice
3         6 2024-01-19    75.0     bob

order_id      int64
order_date  datetime64[ns]
amount      float64
customer      object
dtype: object

```

Twelve lines of pandas to parse five different kinds of mess: mixed date formats, dollar signs, thousands commas, stray whitespace, inconsistent case, fully-empty rows, missing amounts. This is what the first cell of essentially every analysis notebook looks like.

8.9 Exercises

1. Given a DataFrame with a column `email` that contains strings like `"alice@example.com"`, `"BOB@EXAMPLE.COM"`, and `" carol@example.com "`, write a one-liner that normalizes them to lowercase, strips whitespace, and splits into `user` and `domain` columns.
2. You have a DataFrame with `NaN` in several columns. Write code that prints a table with one row per column, showing the count of missing values and the percentage of the DataFrame they represent. Sort by count descending.
3. Rewrite this loop-y code as a chained pandas expression:

```

out = []
for _, row in df.iterrows():
    if row["amount"] > 0:
        out.append({"customer": row["customer"].lower(),
                    "amount": row["amount"] * 1.1})

```

Why is the chained version faster?

4. `.fillna(method="ffill")` is sometimes the wrong fill for time series — for example if a gap spans a boundary where you wouldn't expect carry-over. Read the docs for the `limit=` argument and describe, in one sentence, what it does.

5. You have a column `id` that contains strings like `"ORD-0012"` and you want just the integer portion. Write it three ways: with `.str.split`, with `.str.extract(r"(\d+)")`, and with `.str.slice`. Which do you prefer? Why?

Chapter 9 is the last pandas chapter — and the most rewarding. It is about the three operations that are the whole point of using a DataFrame: groupby, join, and reshape.

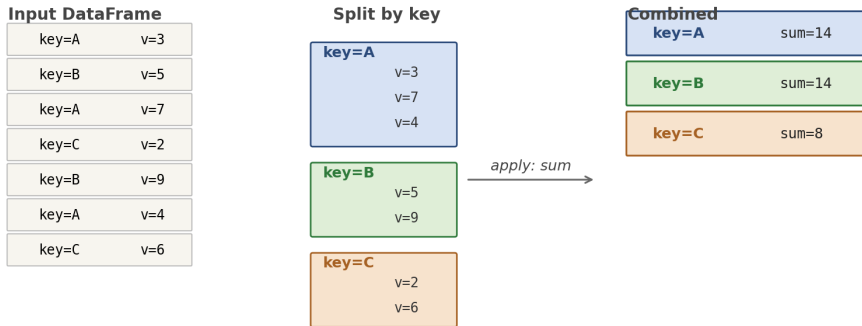
CHAPTER

Grouping, Joining, Reshaping

If there's a pandas chapter that repays rereading, this is it. The three operations in its title — groupby, merge, reshape — are the whole point of moving to a DataFrame from a pile of NumPy arrays. I'll spend most of the space on groupby, since it's where most analyses live.

9.1 Split–apply–combine

`df.groupby(key)` splits a DataFrame into groups, applies a function to each one, and combines the results. The pattern is so pervasive that the phrase *split–apply–combine* is a term of art.

groupby: split → apply → combine

`df.groupby("key")["v"].sum()` — three lines of pandas, one C loop underneath.

groupby in pictures: split rows into groups by a key column, apply an aggregation to each group independently, then combine the results back into a smaller DataFrame. Under the hood this is a single pass over the input — pandas doesn't build the intermediate split DataFrames explicitly.

The minimal example:

```
import pandas as pd

df = pd.DataFrame({
    "key": ["A", "B", "A", "B", "C", "A"],
    "v":   [1, 2, 3, 4, 5, 6],
})

df.groupby("key")["v"].sum()
```

```
key
A    10
B     6
C     5
Name: v, dtype: int64
```

Any of these work as the aggregation:

```
df.groupby("key")["v"].mean()
df.groupby("key")["v"].count()
df.groupby("key")["v"].min()
df.groupby("key")["v"].max()
df.groupby("key")["v"].std()
```

You can group by multiple keys:

```
df.groupby(["region", "product"])["sales"].sum()
```

The result has a two-level (MultiIndex) row label — `(region, product)`. `reset_index()` flattens it back into columns.

Named aggregations

For a real analysis, you usually want several aggregations at once, with informative names:

```
df.groupby("key").agg(
    total = ("v", "sum"),
    n = ("v", "count"),
    mean = ("v", "mean"),
    max = ("v", "max"),
)
```

	total	n	mean	max
key				
A	10	3	3.33	6
B	6	2	3.00	4
C	5	1	5.00	5

Each keyword argument is `name=(column, function)`. The function can be a string name (`"sum"`, `"mean"`, ...) or a callable. This is my favourite pandas idiom — it produces a tidy, named summary in one call.

`transform` **and** `apply`

`.agg` returns one row per group. `.transform` returns the same shape as the input — each element gets the value its group would have computed. Handy for normalising within groups:


```
df["v_zscore"] = (
    df["v"]
    .groupby(df["key"])
    .transform(lambda s: (s - s.mean()) / s.std(ddof=0))
)
```

`.apply` is the catch-all. Its function receives each group as a whole DataFrame and returns whatever; pandas stitches the results back together. Reach for it only when `.agg` / `.transform` won't do, because it's slower and harder to reason about.

9.2 Joining: `merge` vs `concat`

`merge`

SQL-style joins. The signature you need to remember is:

```
pd.merge(left, right, on="id", how="inner")
```

`how` is one of `"inner"`, `"left"`, `"right"`, `"outer"`, or `"cross"`. All behave the way their SQL namesakes do.

```
a = pd.DataFrame({"id": [1, 2, 3], "x": [10, 20, 30]})
b = pd.DataFrame({"id": [2, 3, 4], "y": ["a", "b", "c"]})

a.merge(b, on="id")                                # inner
#   id  x  y
# 0  2 20  a
# 1  3 30  b

a.merge(b, on="id", how="left")
#   id  x  y
# 0  1 10 NaN
# 1  2 20  a
# 2  3 30  b
```

Three arguments you'll reach for:

```
a.merge(b, left_on="cust_id", right_on="id")      # different key names
a.merge(b, on="id", suffixes=("_a", "_b"))        # disambiguate collisions
a.merge(b, on=["id", "date"])                    # composite key
```

Warning

Always check the row count before and after a merge. If you join on a key that is not unique in `right`, `merge` fans out and you end up with more rows than you started with — sometimes much more. Pass `validate="one_to_one"` (or `"one_to_many"`) when you know what to expect, and pandas will raise if the shape is wrong.

concat

Stack DataFrames. The default is vertical (`axis=0` , stack rows):

```
pd.concat([df_q1, df_q2, df_q3, df_q4])
```

`axis=1` stacks columns side by side. `ignore_index=True` resets the row index — you almost always want it when concatenating independent tables.

```
pd.concat([a, b], ignore_index=True)
```

9.3 Reshape: long vs wide

The same data can live in two layouts:

Wide: one row per observation, one column per measurement.

	date	seoul	tokyo	osaka
0	2024-01-01	-3.2	4.1	3.5
1	2024-01-02	-2.0	3.8	2.9

Long: one row per measurement, one column for the variable name.

	date	city	temp_c
0	2024-01-01	seoul	-3.2
1	2024-01-01	tokyo	4.1
2	2024-01-01	osaka	3.5
3	2024-01-02	seoul	-2.0
...			

Long is better for analysis (groupby, plots with hue, statistical models). Wide is better for display. Pandas has two functions that translate between them.

Long → wide with `pivot`

```
wide = long.pivot(index="date", columns="city", values="temp_c")
```

Wide → long with `melt`

```
long = wide.melt(id_vars="date",
                 var_name="city",
                 value_name="temp_c")
```

`pivot_table` is `pivot` plus an aggregation — useful when the same (index, columns) pair appears more than once and you need to reduce:

```
wide = long.pivot_table(index="date", columns="city",
                        values="temp_c", aggfunc="mean")
```

9.4 Rolling windows

For time series and running statistics:

```
s = pd.Series(
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    index=pd.date_range("2024-01-01", periods=10),
)

s.rolling(window=3).mean()
# 2024-01-01    NaN
# 2024-01-02    NaN
# 2024-01-03    2.0
# 2024-01-04    3.0
# ...
```

The first `window - 1` entries are `NaN` because there isn't yet a full window. Pass `min_periods=1` to relax that:

```
s.rolling(window=3, min_periods=1).mean()
```

Other useful window operations: `.rolling().sum()` , `.std()` , `.quantile(q)` , `.apply(custom_func)` . `.expanding()` is a cumulative version; `.ewm()` exponentially-weighted.

9.5 Time series resampling

`resample` is `groupby` for time:

```
daily = s.resample("D").mean()      # daily mean
weekly = s.resample("W").sum()      # weekly sum
monthly_ohlc = s.resample("M").ohlc() # open, high, low, close
```

The frequency strings are well documented: `D` day, `H` hour, `W` week, `M` month-end, `Q` quarter, `Y` year, `10min` ten-minute, and so on.

`shift` moves values forward or backward in time:

```
s.shift(1)      # yesterday's value aligned to today
s - s.shift(1)  # first differences
```

9.6 A full example

One pipeline, combining everything so far. Input: a long-format sales table. Goal: a monthly summary by region, with growth vs. the previous month.

```

sales = pd.DataFrame({
    "date": pd.to_datetime(["2024-01-05", "2024-01-18", "2024-02-03",
                           "2024-02-15", "2024-03-10", "2024-03-22"]),
    "region": ["APAC", "EMEA", "APAC", "EMEA", "APAC", "EMEA"],
    "amount": [120, 85, 140, 90, 160, 95]
})

monthly = (
    sales
    .assign(month = lambda d: d["date"].dt.to_period("M"))
    .groupby(["month", "region"], as_index=False)
    .agg(revenue = ("amount", "sum"))
    .sort_values(["region", "month"])
)

monthly["mom_growth_pct"] = (
    monthly
    .groupby("region")["revenue"]
    .pct_change()
    .mul(100)
    .round(1)
)

print(monthly.to_string(index=False))

```

month	region	revenue	mom_growth_pct
2024-01	APAC	120	NaN
2024-02	APAC	140	16.7
2024-03	APAC	160	14.3
2024-01	EMEA	85	NaN
2024-02	EMEA	90	5.9
2024-03	EMEA	95	5.6

Ten lines of pandas. Without a library, this is 40–50 lines of nested loops and dictionaries and it would still be slower.

9.7 Exercises

1. Given a DataFrame of sales with columns `date`, `product`, and `amount`, produce the total revenue per product per month, sorted by month and then by revenue descending. (Hint: `groupby` with two keys, then `sort_values` with two keys.)

2. Using the daily Series from Section 9.4, compute a 7-day rolling standard deviation. What does the first week look like?
3. Reshape the `sales` DataFrame from Section 9.6 from long to wide — one row per month, one column per region, values are revenue. Then reshape back to long. Do you get the original DataFrame?
4. You have two DataFrames, `orders` (with `customer_id`) and `customers` (with `id`, `country`). Compute total revenue per country with a single chained expression.
5. A common bug: you do `pd.merge(a, b, on="id")` and the result has twice as many rows as `a`. Diagnose: what property of `b` caused it? How would `validate=...` have caught it?

You now have the full pandas working vocabulary. Part IV moves to matplotlib, which is the last major piece of the stack and the one that turns your analyses into things other people can see.

CHAPTER

matplotlib Fundamentals

You've been using matplotlib since Chapter 1. This chapter is the part where we stop using it and start *understanding* it, because whenever your plot does something you didn't want, the fastest way out is to know what the library is actually doing.

Matplotlib has two APIs that call the same underlying code with different style. The `pyplot` style is the one that looks like MATLAB (`plt.plot(...)`, `plt.title(...)`, implicit current figure). The *object-oriented* style explicitly creates a `Figure` and an `Axes` and calls methods on them. This chapter uses the object-oriented style from now on, because it scales to grids of subplots and to interactive notebooks.

10.1 Figure, Axes, Artist

Three terms carry most of the weight:

- **Figure** — the whole canvas. There is one per `plt.subplots()` call. It holds one or more Axes and a few global things (suptitle, legend).
- **Axes** — one coordinate system. Almost every method you call is on an Axes: `ax.plot`, `ax.set_xlabel`, `ax.legend`. Confusingly, "axes" (plural form of "axis") in the English sense is a separate thing — `ax.xaxis`, `ax.yaxis`.
- **Artist** — everything drawn on an Axes: lines, text, legend, spines, ticks, patches, images. Every plotting call creates or mutates artists.

Figure

the outermost container

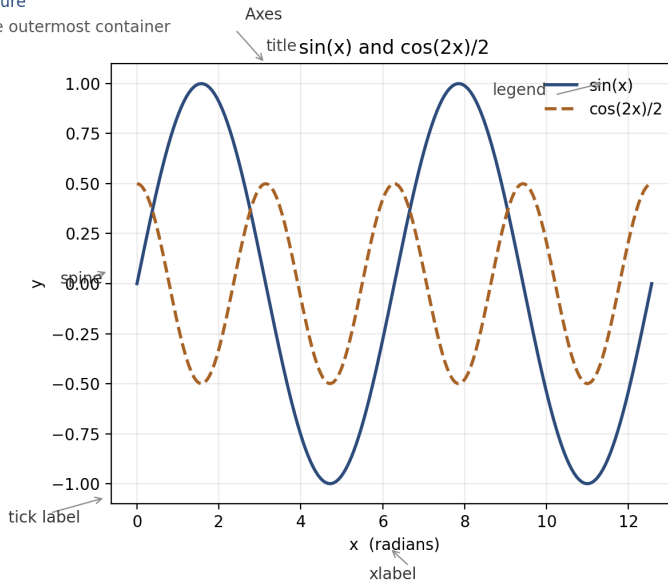


Figure:

one canvas.

Axes:

one coordinate system.

A Figure can contain many.

Artists:

everything drawn on an Axes —
lines, text, legend, spines,
ticks, patches, images.

Every plotting call

creates or mutates artists.

The parts of a matplotlib figure. The outermost Figure is a canvas; an Axes is one coordinate system inside it. Everything drawn on the Axes is an Artist: the lines you plotted, the title, the tick labels, the spines. Every plotting method on an Axes returns one or more artists you can modify later.

The canonical way to make a figure:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)

fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(x, np.sin(x), label="sin")
ax.plot(x, np.cos(x), label="cos")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Two trig functions")
ax.legend()
fig.savefig("trig.png", dpi=150, bbox_inches="tight")
```

Six lines to the plot, two to save. The pattern — `fig, ax = plt.subplots(); ax.plot(...); ...` — is the one to internalize. You'll write it many thousands of times.

10.2 Grids of Axes

`plt.subplots(nrows, ncols, ...)` gives you a 2-D NumPy array of Axes:

```
fig, axes = plt.subplots(2, 3, figsize=(9, 5), constrained_layout=True)
for i, ax in enumerate(axes.flat):
    ax.plot(np.random.default_rng(i).standard_normal(100).cumsum())
    ax.set_title(f"seed {i}")
```

Three arguments worth knowing:

- `figsize=(w, h)` — size in inches. Publication plots are typically 6–8 inches wide.
- `constrained_layout=True` — pandas-style automatic layout that prevents overlap. Prefer this to the older `plt.tight_layout()`.
- `sharex=True, sharey=True` — subplots share the same axis limits and the outer tick labels disappear. Great for small multiples.

Tip

`axes.flat` iterates over all the Axes regardless of grid shape — one for loop works for any `(nrows, ncols)`. `axes.ravel()` does the same.

10.3 Plot types you'll use constantly

Line

```
ax.plot(x, y)
ax.plot(x, y, color="#2b4a7a", linestyle="--", linewidth=2,
        marker="o", markersize=4, label="series 1")
```

Line styles: `'-'` solid (default), `'--'` dashed, `'-.'` dash-dot, `':'` dotted. Colors can be named (`"red"`), single-letter (`"r"`), hex (`"#2b4a7a"`), or RGB tuples in `[0, 1]` .

Scatter

```
ax.scatter(x, y, c=labels, cmap="viridis",
           s=sizes, alpha=0.7, edgecolor="white")
```

`c=` accepts an array for color-by-variable; pair with `cmap` . `s=` is marker size (or a per-point array). `alpha` for transparency — essential when points overlap.

Bar

```
ax.bar(categories, heights, color="#2b4a7a")
ax.barh(categories, heights) # horizontal
```

For grouped bars, plot each group at shifted x positions:

```
x = np.arange(len(labels))
w = 0.35
ax.bar(x - w/2, values_a, width=w, label="A")
ax.bar(x + w/2, values_b, width=w, label="B")
ax.set_xticks(x, labels)
```

Histogram

```
ax.hist(data, bins=30, edgecolor="white")
```

`bins` is either a number or an explicit list of edges. `density=True` normalizes to a PDF. `cumulative=True` makes a CDF.

Image

```
ax.imshow(matrix, cmap="coolwarm", aspect="auto")
fig.colorbar(mappable, ax=ax)
```

`imshow` displays a 2-D array as a colored grid. Pair with `colorbar` when the color encodes a value.

10.4 Colors, styles, and rcParams

Matplotlib has an enormous number of global defaults. They live in `matplotlib.rcParams`, a dictionary that you can mutate:

```
import matplotlib as mpl

mpl.rcParams["font.family"] = "Helvetica Neue"
mpl.rcParams["axes.spines.top"] = False
mpl.rcParams["axes.spines.right"] = False
mpl.rcParams["axes.grid"] = True
mpl.rcParams["grid.alpha"] = 0.25
```

For larger sets of defaults, put them in a `.mplstyle` file and load with `plt.style.use(path)`. Chapter 12 is about building one.

Built-in styles cover most of what you'd reach for:

```
plt.style.available
# ['Solarize_Light2', 'bmh', 'dark_background', 'default',
#  'fast', 'ggplot', 'grayscale', 'seaborn-v0_8', ...]
plt.style.use("ggplot")
```

10.5 Saving for print vs. screen

Three knobs you'll adjust every time:

```
fig.savefig("out.png", dpi=200, bbox_inches="tight")
fig.savefig("out.pdf")                # vector, perfect
fig.savefig("out.svg")                # vector, for the web
```

PNG with high DPI (≥ 200) is the default for reports and slides. **PDF** is vector — infinite resolution, small for simple plots, ideal for papers. **SVG** is vector-for-web.

`bbox_inches="tight"` clips the output to the actual figure content, cropping empty margins. You want it almost every time.

Warning

Raster plots (PNG, JPG) at the wrong DPI will pixelate when printed. For anything that's going to a printer or a PDF, either use vector output (`.pdf`) or bump the DPI to at least 300.

10.6 The closest thing to a universal plot idiom

When I want a clean plot in a hurry, this is what I type:

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, y, color="#2b4a7a", linewidth=2)
ax.set_xlabel("x", color="#555")
ax.set_ylabel("y", color="#555")
ax.set_title("Title", fontweight="bold", loc="left", color="#222")
ax.spines[["top", "right"]].set_visible(False)
ax.spines[["left", "bottom"]].set_color("#aaa")
ax.tick_params(colors="#555", length=3)
ax.grid(axis="y", linestyle=":", color="#ccccc", alpha=0.8)
fig.tight_layout()
```

Four things that lift it above defaults: (1) left-aligned title that doesn't compete with the curve for center attention, (2) spines on only two sides, (3) muted gridlines on the y-axis only, (4) consistent muted colors for axis labels and ticks. These cost nothing and they consistently make plots look better.

10.7 Exercises

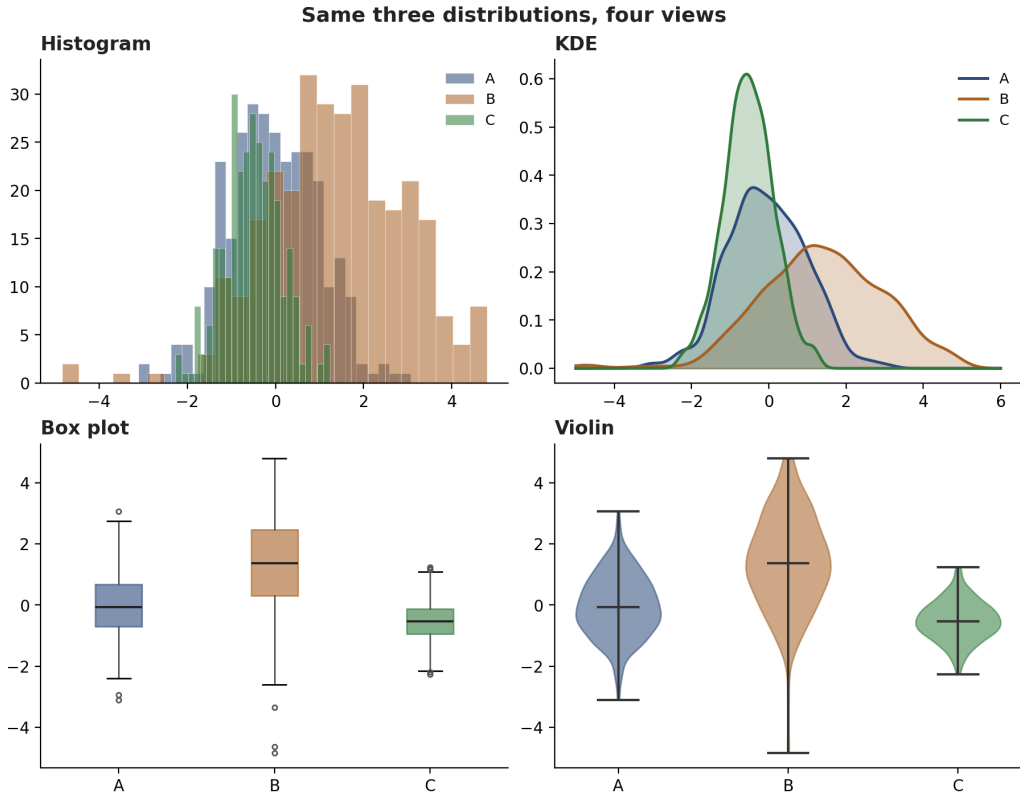
1. Using a single `plt.subplots(2, 2)` call, produce four plots of $f(x) = \sin(k \cdot x)$ for `k = 1, 2, 3, 4`. Share the x-axis; do not share the y-axis.
2. The `x` axis of a bar plot with long categorical labels overlaps. Three fixes: rotate the labels (`ax.tick_params`), make the figure wider, or use `barh` instead. Try all three. Which do you prefer?
3. Take any Axes object and list every method that starts with `set_`. (`dir(ax)` or `ax.__dir__()`.) How many are there?
4. Compare `plt.tight_layout()` and `constrained_layout=True` on a 2x3 grid of plots with varying title lengths. Which behaves better when title lengths vary between subplots?
5. Create a figure, save it three times: as PNG at DPI 100, PNG at DPI 300, and PDF. Open each. For what use case is each the right choice?

Chapter 11 covers the plot types you reach for specifically to look at distributions.

CHAPTER

Statistical Plots

When the data is a list of measurements — a column of heights, a batch of model scores, a set of response times — the question you are usually trying to answer is "what does this distribution look like?" There are four plot types that give useful answers, and knowing which one to reach for is the point of this chapter.



The same three distributions plotted four ways. A histogram is honest about bin counts but obscured by choice of bin width. A KDE (kernel density estimate) smooths the histogram but hides the raw counts. A box plot shows quantiles but collapses the shape. A violin plot combines the KDE with the box plot's quantiles and is usually the most informative choice when you're comparing distributions side by side.

11.1 Histograms

A histogram counts how many observations fall into each bin:

```
ax.hist(data, bins=30, edgecolor="white", color="#2b4a7a")
ax.set_xlabel("value"); ax.set_ylabel("count")
```

The number of bins matters. Too few and you lose structure; too many and you see noise. As a rule of thumb, start with `bins=30` and adjust if the shape looks wrong. Matplotlib also accepts an array of bin edges for full control.

For comparing two distributions on the same axis:

```
ax.hist(group_a, bins=30, alpha=0.55, label="A", edgecolor="white")
ax.hist(group_b, bins=30, alpha=0.55, label="B", edgecolor="white")
ax.legend()
```

`alpha=0.55` is the magic number — transparent enough to see overlap, opaque enough to read color separately. With more than three groups a histogram gets busy; switch to a KDE or a violin plot.

Two useful variants:

```
ax.hist(data, density=True)           # normalized so the area is 1
ax.hist(data, cumulative=True)        # CDF, not PDF
```

11.2 KDE — kernel density estimates

A KDE smooths a histogram by placing a small bell curve on every data point and summing them. The result is continuous and insensitive to bin choice:

```
from scipy.stats import gaussian_kde

kde = gaussian_kde(data, bw_method=0.25)
xs = np.linspace(data.min(), data.max(), 400)
ax.plot(xs, kde(xs), color="#2b4a7a", linewidth=2)
ax.fill_between(xs, kde(xs), color="#2b4a7a", alpha=0.2)
```

The `bw_method` argument controls the bandwidth — how wide each bell is. Small bandwidth = spiky curve that tracks every bump; large bandwidth = smooth curve that might hide real features. Scott's rule (the default) is fine for most data; override it only when the default looks wrong.

Note

If you're doing a lot of distribution plotting, consider `seaborn.kdeplot` or `seaborn.histplot`. They are built on `matplotlib` and do the common cases (overlay, hue, rug) with one line. This book sticks to `matplotlib` to keep dependencies small.

11.3 Box plots

A box plot shows five numbers: the median, the interquartile range (25th and 75th percentiles), and the "whiskers" (conventionally $1.5\times$ the IQR from each quartile). Outliers beyond the whiskers appear as individual points.

```
ax.boxplot([group_a, group_b, group_c], tick_labels=["A", "B", "C"])
```

Box plots are great when you have many groups (five, ten, fifty) and want a compact side-by-side comparison. They hide shape — two distributions with the same five-number summary but very different modalities look identical.

If you need to keep the outliers but get them to stop dominating the y-axis, try `ax.boxplot(..., showliers=False)` or use a violin plot instead.

11.4 Violin plots

A violin plot is a KDE mirrored around a vertical axis, usually with the box-plot summary drawn inside it. It combines the shape information of a KDE with the quantile summary of a box plot:

```
parts = ax.violinplot([group_a, group_b, group_c], showmedians=True)
for pc, color in zip(parts["bodies"], ["#2b4a7a", "#a66123", "#2f7a3b"]):
    pc.set_facecolor(color); pc.set_alpha(0.6); pc.set_edgecolor(color)
ax.set_xticks([1, 2, 3]); ax.set_xticklabels(["A", "B", "C"])
```

When I have 2–8 groups and want the reader to see both shape and summary statistics, violin is usually the right choice.

11.5 Scatter with error

For a measurement of a response (`y`) at several inputs (`x`) with known uncertainty:

```
ax.errorbar(x, y, yerr=y_err, fmt="o",
            color="#2b4a7a", ecolor="#888",
            capsize=3, markersize=5, linewidth=1)
```

`fmt="o"` draws markers only (no connecting line). `yerr` can be a scalar, a 1-D array of per-point half-widths, or a 2-D array of (lower, upper) bounds.

For a continuous confidence band under a curve:

```
ax.plot(x, y_hat, color="#2b4a7a")
ax.fill_between(x, y_hat - ci, y_hat + ci,
               color="#2b4a7a", alpha=0.2)
```

`fill_between` is the workhorse for confidence regions, training/val loss bands, and any other "area between two curves" situation.

11.6 Scatter matrices

When you want to see every pair of features at once — which is what we did in Chapter 1 — you build a `(d, d)` grid of scatter plots with histograms on the diagonal. pandas has one built in:

```
from pandas.plotting import scatter_matrix
scatter_matrix(df, figsize=(9, 9), alpha=0.6, diagonal="kde")
```

For three to six features this is a useful survey tool. For more, the figure gets busy and you should filter down or use a correlation heatmap instead:

```
corr = df.corr()
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
fig.colorbar(im, ax=ax)
ax.set_xticks(range(len(corr))); ax.set_xticklabels(corr.columns, rotation=45, ha="r")
ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns)
```

11.7 Choosing a plot

The decision tree I actually use:

- **One distribution, < 1000 points.** Histogram (30 bins).
- **One distribution, ≥ 1000 points.** KDE or histogram with many bins.
- **2–8 distributions to compare.** Violin plot.
- **≥ 10 distributions to compare.** Box plots (small multiples if groups have structure).
- **Two variables with uncertainty.** Errorbar or `fill_between`.
- **Every pair of features in a small dataset.** Scatter matrix.

- **Correlations in a larger dataset.** Heatmap.

When in doubt, try two and show a colleague. The plot that prompts the most questions is usually the one you want.

11.8 Exercises

1. Generate 1,000 samples each from two distributions of your choice. Produce the same data as a histogram, a KDE, a box plot, and a violin plot. Which reads best?
2. Read the docs for `ax.hist`'s `histtype` parameter. What does `"step"` do differently from the default? When is it better?
3. Build a correlation heatmap for the iris dataset (four features). Is any pair of features more correlated than the others? Does that match what you saw in the scatter matrix from Chapter 1?
4. Given a noisy measurement $y = \sin(x) + \varepsilon$, plot the true function, the observations, and a 95% confidence band based on a rolling standard deviation. Use `fill_between`.
5. A violin plot shows the full shape, but people who rarely see them find the "fat ends" confusing (it's a KDE tail that was never observed). Read the docs for `showextrema`, `showmedians`, and `quantiles`, and build a version that omits the mirrored tails below the minimum and above the maximum.

Chapter 12 is about judgement: how to pick scales, when to annotate, when to use small multiples, and how to pack one or two figures' worth of design decisions into a style sheet you can reuse forever.

CHAPTER

Designing Figures that Communicate

A plot is communication. If the reader can glance at it and get the point in three seconds, the plot is doing its job. If they have to squint, cross-reference the legend, and re-read the axis

labels, they'll skip it — and with it, your argument. This chapter is about the small set of decisions that make the difference.

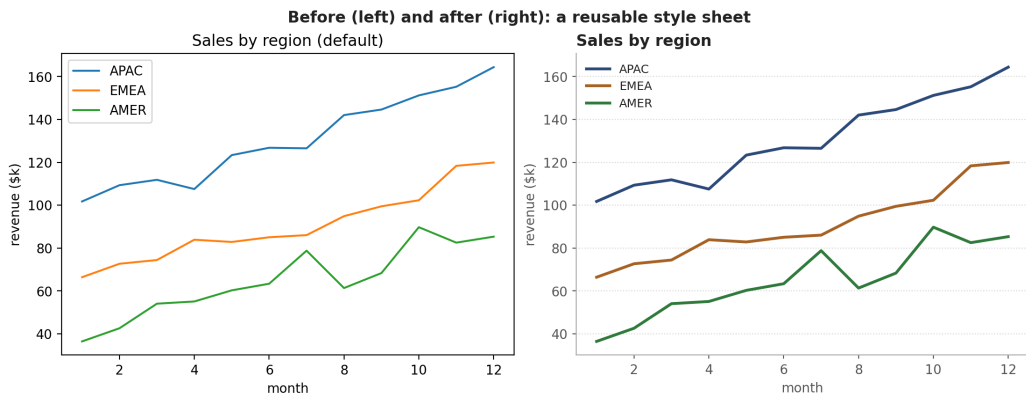
12.1 The four things readers see first

In order of how much of their attention they get:

1. **Shape.** The curves, bars, or points. If this doesn't communicate the key finding on its own, no amount of labelling will save it.
2. **Title.** Usually on the top-left; should say *what the figure shows*, not the axis names.
3. **Axis labels.** What do `x` and `y` mean, in what units.
4. **Legend.** Who is who, when there are multiple series.

Everything else — tick spacing, gridlines, colors — is second-order. Get those four right and most plots are already fine.

12.2 Before and after: a reusable style sheet



Left: matplotlib defaults. Right: the same plot after applying a small, reusable style sheet. The styled version isn't "prettier" arbitrarily — every change is a direct consequence of a deliberate choice about what the reader should see first. The curves are bolder; the gridlines are demoted; the title is left-aligned; tick labels are muted; spines are removed where they don't carry information.

Here is the styling code, at length:

```
def nice_axis(ax):
    ax.spines[["top", "right"]].set_visible(False)
    ax.spines[["left", "bottom"]].set_color("#aaa")
    ax.tick_params(colors="#555", length=3)
    ax.grid(axis="y", linestyle=":", color="#cccccc", alpha=0.8)
    if ax.get_title():
        ax.set_title(ax.get_title(), fontweight="bold",
                     color="#222", loc="left", fontsize=12)
    ax.xaxis.label.set_color("#555")
    ax.yaxis.label.set_color("#555")
```

The same effect is available globally via a style sheet. Create `mystyle.mplstyle` :

```
axes.spines.top:      False
axes.spines.right:    False
axes.edgecolor:       #aaaaaa
axes.titleweight:     bold
axes.titlelocation:   left
axes.titlecolor:      #222222
axes.labelcolor:      #555555
xtick.color:          #555555
ytick.color:          #555555
grid.color:           #cccccc
grid.linestyle:       :
grid.alpha:           0.8
axes.grid:            True
axes.grid.axis:       y
```

Then `plt.style.use("mystyle.mplstyle")` at the top of a notebook, and every subsequent plot inherits these defaults. You only need to tune one file and it propagates everywhere.

12.3 Choose axes before you draw

Four choices, made up front, that save pain later.

Linear vs. log

Any quantity that spans multiple orders of magnitude — file sizes, counts over time, distributions with long tails — wants a log scale:

```
ax.set_yscale("log")
```

A rule of thumb: if your smallest value and your largest value differ by more than a factor of 30, try log. If the curve then looks approximately straight, you know you're looking at exponential growth or a power law.

Fixed vs. shared limits

For small multiples — many subplots that you want the reader to compare — share the axes:

```
fig, axes = plt.subplots(2, 3, sharex=True, sharey=True)
```

Shared axes make differences between panels meaningful. If one panel shows $y \in [0, 10]$ and another $y \in [0, 1000]$ with their own scales, the eye cannot compare them at all.

Start y at zero (when it matters)

For bar charts and other plots where the magnitude carries meaning, start the y-axis at zero. Anything else exaggerates differences. For time series where you care about relative change, starting at zero wastes space, so don't.

Explicit number formatting

```
from matplotlib.ticker import FuncFormatter, PercentFormatter

ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _: f"${v:,.0f}"))
# or
ax.yaxis.set_major_formatter(PercentFormatter(1.0)) # expects 0..1
```

Dollar signs, commas, percent symbols, K/M/B suffixes — all belong on the ticks, not in the axis label. "Revenue (\$)" with tick labels like "1000000" is harder to read than unlabelled ticks reading "\$1M".

12.4 Annotate the point

If you're telling a story with the figure, highlight the thing the story is about:

```
ax.annotate("launch",
            xy=(launch_date, launch_value),
            xytext=(launch_date, launch_value * 1.4),
            arrowprops=dict(arrowstyle="->", color="#555"),
            fontsize=9, color="#222")
```

Two moderation rules. First, one annotation per figure, at most three. A figure isn't a book; crowd it and the reader gives up. Second, the annotation should read on its own — "launch" is fine, "event 1" is not.

For a band of interest:

```
ax.axvspan(start, end, color="#f0e6d6", alpha=0.5, zorder=0)
ax.axhline(y=threshold, color="#888", linestyle="--", linewidth=1)
```

`zorder=0` pushes the span behind the data — otherwise it covers your curves. `axvline` and `axhline` are for vertical and horizontal reference lines.

12.5 Small multiples

When you have a variable with several categories and you're tempted to plot all of them on the same axis with different colors, try small multiples instead — one subplot per category, same axes:

```
categories = df["category"].unique()
fig, axes = plt.subplots(1, len(categories),
                        sharex=True, sharey=True,
                        figsize=(3 * len(categories), 3))
for ax, cat in zip(axes, categories):
    sub = df[df["category"] == cat]
    ax.plot(sub["x"], sub["y"])
    ax.set_title(cat, fontsize=10)
```

Small multiples trade color coding for spatial separation. The result is a figure that works with four categories as well as with sixteen, and where differences across categories are perceived through position rather than hue.

12.6 Color, for the reader (not for the author)

Three rules do 90% of the work.

Use a qualitative palette for categories

Default matplotlib colors are fine. Tableau's 10-color palette is better. `"tab:blue"`, `"tab:orange"`, `"tab:green"`, ..., are the named Tableau colors and they've been optimized for this.

Use a sequential colormap for ordered data

`"viridis"`, `"plasma"`, `"cividis"`. They are perceptually uniform (equal increments look equal), colorblind-safe, and print to grayscale sensibly. Do not use `"jet"` — it's visually exciting and perceptually dishonest.

Use a diverging colormap for signed data

When zero is a meaningful midpoint (correlations, residuals, temperature anomalies), use a diverging map with `vmin=-vmax` :

```
vmax = max(abs(data.min()), abs(data.max()))
ax.imshow(data, cmap="coolwarm", vmin=-vmax, vmax=vmax)
```

A diverging map that isn't centered at zero lies about where the balance point is.

12.7 Saving for the medium

The last step of any figure is the one most easily forgotten.

For slides and web: PNG at 150–200 DPI, whatever width the presentation calls for.

For papers and reports: PDF, and set the `figsize` to something close to the column width of the target document (usually 3.3 inches for a single-column paper, 6.5 inches for a full-width figure). Save with `bbox_inches="tight"` .

For interactive dashboards: SVG, which scales without pixellation. Wrap in HTML.

Pick one target, tune the figure for it, and commit. Plots that look fine on a laptop screen are often illegible when projected, and plots that look sparse on a large screen are often busy on a phone.

12.8 Exercises

1. Take any figure from an earlier chapter and rewrite the code to produce a styled version using the `nice_axis` function from Section 12.2. Do you find the styled version easier to read? Harder?
2. Write the `mystyle.mplstyle` file from Section 12.2 to disk and load it at the top of a fresh notebook. Plot anything. Describe what changed relative to defaults.
3. A figure has four curves. When, if ever, should you use four different colors? Four different line styles? Four small multiples? Justify each choice.
4. Find a published figure you think is bad. Describe, in three bullets, what's wrong with it and what you'd change. (This is the single most effective exercise in the chapter — habit-forming.)
5. Read the docs for `matplotlib.colormaps`. Which colormaps are perceptually uniform? Which are not? For a heatmap of correlations, which would you use, and why?

Part IV is done. Part V turns to scikit-learn — where you stop describing data and start building models with it.

CHAPTER

Statistics in Code

Statistics is its own subject. This chapter is not the subject; it is the code you need to do the minimum honest thing with a numerical summary. The goal is that when someone asks "are these two groups actually different?" you can answer with more than a difference of means and a hopeful expression.

13.1 Descriptive statistics

Three pandas methods cover most of what you will ever need:

```
import pandas as pd, numpy as np

s = pd.Series(np.random.default_rng(0).normal(5, 2, 100))

s.describe()
# count    100.000000
# mean      4.878...
# std       2.012...
# min       0.060...
# 25%       3.526...
# 50%       4.875...
# 75%       6.183...
# max       9.687...
```

For a weighted or robust summary:

```
s.quantile([0.025, 0.5, 0.975])    # 95% range
s.median()                          # robust to outliers
s.mad() if hasattr(s, 'mad') else (s - s.median()).abs().median()
```

`skew()` and `kurt()` tell you whether a distribution is long-tailed or heavy. They're diagnostic, not primary; reach for them when a histogram already suggests trouble.

13.2 Distributions from `scipy.stats`

`scipy.stats` has every distribution you've ever heard of and a uniform API for all of them. Pick one, freeze its parameters, then `.pdf`, `.cdf`, `.ppf`, `.rvs`:

```
from scipy import stats

d = stats.norm(loc=5, scale=2)          # N(5, 4)
d.pdf(5)                               # density at 5
d.cdf(7)                               # P(X <= 7)
d.ppf(0.975)                           # 97.5th percentile (inverse CDF)
d.rvs(size=10, random_state=0)         # 10 random samples
```

Swap `norm` for `t`, `binom`, `poisson`, `beta`, `gamma`, `expon`, `uniform` — the API is identical.

Three operations I reach for often:


```
# 95% two-sided critical value of a standard normal
stats.norm().ppf(0.975)          # 1.959...

# Probability of a binomial 5/10 or more extreme (two-sided p-value)
2 * stats.binom(10, 0.3).sf(4)    # sf = 1 - cdf, for upper tails

# Fit parameters of an empirical distribution to a candidate family
mu, sigma = stats.norm.fit(data)
```

`fit` gives you max-likelihood estimates. It's the fastest way to check "is this approximately normal, roughly?" — if the histogram and the fitted PDF disagree, the answer is no.

13.3 Confidence intervals, two ways

By formula (for means)

Classical textbook CI for the mean of a single sample:

```
def ci_mean(x, alpha=0.05):
    n = len(x); mean = x.mean(); se = x.std(ddof=1) / np.sqrt(n)
    q = stats.t(df=n - 1).ppf(1 - alpha / 2)
    return mean - q * se, mean + q * se
```

Assumes the sampling distribution of the mean is approximately Student-`t`, which is itself a consequence of the central limit theorem. It's fine when the sample is large or the underlying distribution is roughly normal. When neither holds, use the bootstrap.

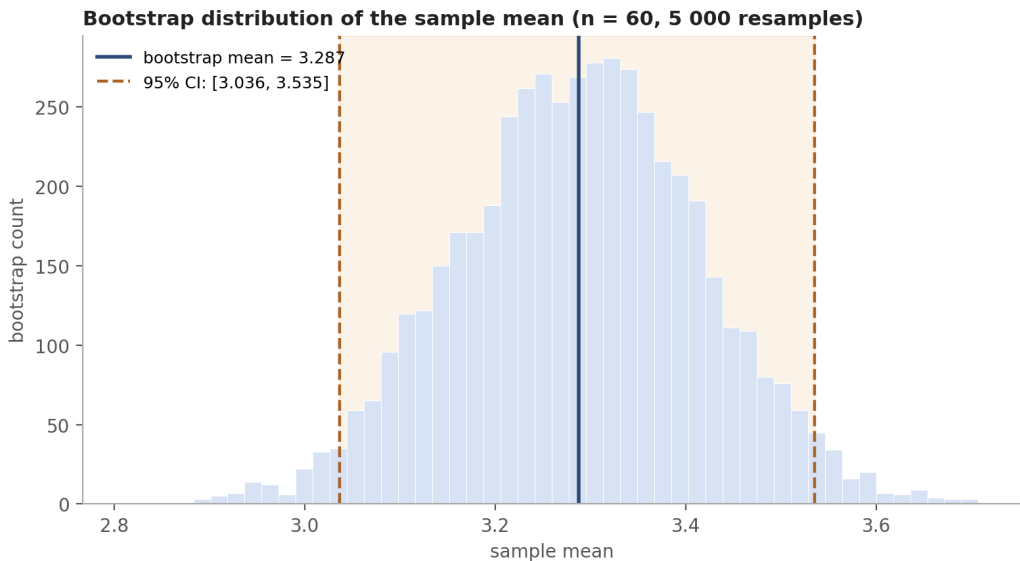
By bootstrap (for anything)

A bootstrap resamples the data *with replacement* many times, computes the statistic on each resample, and takes percentiles of the result. No distributional assumptions; works for any statistic you can compute, not just the mean.

```
def bootstrap_ci(x, stat=np.mean, n_boot=5000, alpha=0.05, rng=None):
    rng = rng or np.random.default_rng()
    idx = rng.integers(0, len(x), size=(n_boot, len(x)))
    est = np.apply_along_axis(stat, 1, x[idx])
    return np.percentile(est, [100 * alpha / 2, 100 * (1 - alpha / 2)])

sample = np.random.default_rng(0).normal(3.2, 1.1, size=60)
lo, hi = bootstrap_ci(sample)
print(f"95% CI for mean: [{lo:.3f}, {hi:.3f}]")
```

95% CI for mean: [3.036, 3.535]



Bootstrap distribution of the sample mean, built from 5 000 resamples of the original 60-element sample. The shaded band is the central 95% of the bootstrap distribution — that's the confidence interval. No assumption about Normality of the underlying population was made.

The bootstrap is my default for any CI where I'm not sure the formulas apply. The only catch is that it needs a representative sample — if your sample is skewed by collection bias, the bootstrap will happily give you a tight CI around the wrong answer.

13.4 Hypothesis tests, demystified

A hypothesis test answers a single question: "if there were no effect, how likely would my data be?" That number is the *p-value*. A small p-value means your data is unlikely under "no effect," which is weak evidence for "there is an effect."

Three tests cover the large majority of situations:

Two-sample t-test

For comparing the means of two independent groups:

```
a = np.random.default_rng(0).normal(0, 1, 50)
b = np.random.default_rng(1).normal(0.5, 1, 50)
result = stats.ttest_ind(a, b, equal_var=False) # Welch
print(f"t = {result.statistic:.3f}, p = {result.pvalue:.4f}")
```

`equal_var=False` gives you Welch's test, which handles unequal variances. Use it; the equal-variance version is more restrictive and rarely better.

Proportion test

For comparing two rates (conversion, click-through, survival):

```
from statsmodels.stats.proportion import proportions_ztest

count = np.array([45, 58])          # successes
n      = np.array([500, 500])       # sample sizes
z, p = proportions_ztest(count, n)
```

The output of `proportions_ztest` is the z-statistic and the p-value for the null hypothesis that the two proportions are equal.

Chi-square for categorical tables

For an $r \times c$ contingency table:

```
table = np.array([[30, 10],
                  [20, 40]])
chi2, p, dof, expected = stats.chi2_contingency(table)
```

Useful for "is the distribution of B different across levels of A?"

13.5 Multiple testing

Run twenty tests at $p < 0.05$ under a true null and expect one false positive just by chance. Correct for this when it matters:

```
from statsmodels.stats.multitest import multipletests
pvalues = [0.01, 0.03, 0.04, 0.2, 0.5]
reject, p_adj, _, _ = multipletests(pvalues, alpha=0.05, method="fdr_bh")
```

`fdr_bh` (Benjamini–Hochberg) controls the false discovery rate and is the right default for exploratory work. `bonferroni` is more conservative.

13.6 A note on practical significance

A p-value is about whether an effect exists, not whether it is big enough to care about. A sample of a million will flag as statistically significant an effect that is well below the noise floor of anything you can act on. Report the effect size alongside the p-value — a percentage difference, a standardized mean difference, something with units — or the reader will conclude the wrong thing.

My personal shorthand, for A/B tests: "7.2% lift with a 95% CI of [2.1%, 12.4%]" is useful. "p = 0.02" is not.

13.7 Exercises

1. Simulate 1,000 samples of size 30 from a `Beta(2, 5)` distribution. Compute a 95% CI for the mean with both the formula from Section 13.3 and the bootstrap. Do they agree? What if you change the sample size to 5?
2. Implement a paired t-test (for before/after measurements) and compare its output with `stats.ttest_rel` for a contrived example.
3. A coin is flipped 50 times and shows 32 heads. Test the null hypothesis that it's fair with: (a) `stats.binom.sf`; (b) a proportion z-test; (c) a chi-square goodness-of-fit test with two cells. Do they agree? Why or why not?
4. Under the null hypothesis of no effect, generate 100 synthetic A/B tests and compute p-values. Plot the histogram. What should it look like? What does it actually look like?

5. Read the docs for `bootstrap_ci` variants — BCa, percentile, basic. The percentile method is what Section 13.3 shows. Under what conditions is BCa better? Under what conditions is it overkill?

Chapter 14 is short: the scikit-learn interface, the single design choice that made the library durable.

CHAPTER

The scikit-learn Interface

scikit-learn's best feature is not any particular model. It is the interface. Every model in the library answers to the same three verbs — `fit`, `predict`, `transform` — so once you know how to use one, you know how to use all of them. This chapter is about that interface and the utilities around it.

14.1 Estimators, transformers, and predictors

Every object you'll interact with in scikit-learn is an *estimator*. An estimator has a `fit(X, y=None)` method that learns something from data.

Two subtypes matter:

Transformers have `transform(X)` — they turn one array into another. `StandardScaler`, `PCA`, `OneHotEncoder`. After fitting, they remember whatever parameters they need to reapply themselves to new data.

Predictors have `predict(X)` — they turn features into predictions. `LogisticRegression`, `RandomForestClassifier`, `Ridge`. Classifiers additionally offer `predict_proba(X)`; regressors don't.

The same pattern holds across every corner of the library:

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Transformers: fit + transform
scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)

pca = PCA(n_components=2).fit(X_train_scaled)
X_pca = pca.transform(X_train_scaled)

# Predictor: fit + predict
model = LogisticRegression().fit(X_pca, y_train)
y_pred = model.predict(pca.transform(scaler.transform(X_test)))
```

The chain in the last line is the whole point: it works because every object speaks the same verbs. Swap in a different scaler or a different classifier — the structure doesn't change.

14.2 `fit_transform` and why it's a lie

A transformer has a `fit_transform(X)` convenience method that does `fit` followed by `transform` in one call:

```
X_scaled = scaler.fit_transform(X_train)
```

The *mistake* is to also call `fit_transform` on your test set:

```
X_test_scaled = scaler.fit_transform(X_test)    # DON'T
```

This refits the scaler to the test set, which means the training and test data live on different scales. Always use `transform` for test data:

```
X_test_scaled = scaler.transform(X_test)
```

The same rule applies to any transformer that learns something from the data — `PCA`, `OneHotEncoder`, `SimpleImputer`, `TargetEncoder`. `fit` on train, `transform` on test.

14.3 Pipelines

Manually juggling fit/transform steps is tedious and error-prone. Scikit-learn has a `Pipeline` that bundles any number of transformers and one final estimator:

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=2)),
    ("clf", LogisticRegression()),
])

pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

A `Pipeline` is an estimator. It exposes `fit`, `predict`, `predict_proba`, `score` — whatever the final estimator exposes — and correctly routes `fit` through every step during training and `transform` during prediction.

`make_pipeline` is a shortcut that names the steps for you by their class names:

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), PCA(n_components=2), LogisticRegression())
```

Tip

Use a pipeline even when your model is a single estimator. The moment you have preprocessing, you want the preprocessing fit-on- train-only behavior for free, plus the ability to pass a single object to cross-validation.

14.4 Training, validation, testing

At minimum, split your data:

```
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y
)
```

`random_state=0` is mandatory — without it your splits are different each run. `stratify=y` keeps class proportions the same across splits, which matters for imbalanced classification.

For hyperparameter tuning you want a third *validation* set or, more commonly, k-fold cross-validation on the training set:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X_tr, y_tr, cv=5, scoring="accuracy")
print(scores.mean(), scores.std())
```

The test set is touched exactly once, at the very end, to estimate how well your final model will generalize. Using it for model selection is called *data leakage* and it's the single most common cause of "the validation score was 0.92 but in production it's 0.74."

14.5 `random_state` and reproducibility

Every estimator that uses randomness — `RandomForestClassifier`, `KMeans`, `train_test_split`, the bootstrap in `BaggingClassifier`, the permutations in `permutation_importance` — takes a `random_state` argument. Pass an integer. Your runs become reproducible; your colleagues' runs match yours; the CI that checks your training output matches your local output.

```
RandomForestClassifier(n_estimators=100, random_state=0)
```

There is no good reason to leave `random_state` unset. The only context where you want *actual* randomness is when you are assessing the variability of your pipeline — and then you set the seed from the trial number, not from `time.time()`.

14.6 Column-by-column preprocessing

Most real tables have mixed types: numeric columns that need scaling, categorical columns that need encoding, date columns that need expansion. `ColumnTransformer` applies different transformers to different subsets of columns:


```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_cols = ["age", "income"]
categorical_cols = ["city", "segment"]

preproc = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
])

pipe = Pipeline([("pre", preproc), ("clf", LogisticRegression())])
pipe.fit(X_train_df, y_train)

```

`ColumnTransformer` accepts column names (works with pandas DataFrames) or integer indices (works with NumPy arrays). The output is a single array with the transformed columns stacked horizontally; you don't usually need to look at it, but `pipe["pre"].get_feature_names_out()` lets you map the output back to readable labels.

14.7 Custom estimators (briefly)

If you need your own transformer, implement `fit(X, y=None)` and `transform(X)` and inherit from `BaseEstimator` and `TransformerMixin`:

```

from sklearn.base import BaseEstimator, TransformerMixin

class LogTransform(BaseEstimator, TransformerMixin):
    def __init__(self, eps: float = 1.0):
        self.eps = eps

    def fit(self, X, y=None):
        return self  # nothing to learn

    def transform(self, X):
        return np.log(X + self.eps)

```

Now `LogTransform(eps=0.5)` works in a `Pipeline` or `ColumnTransformer` like any built-in. This is the extension point that keeps scikit-learn small.

14.8 Exercises

1. Build a pipeline that scales numeric features, one-hot encodes categoricals, and fits a `LogisticRegression`. Verify that `pipe.fit(X_train)` and `pipe.transform(X_test)` give you the same scaling parameters across calls.
2. Without using `Pipeline`, write the fit/transform/predict sequence that it replaces, for a three-step chain. What fit-on-train-only discipline does `Pipeline` enforce that ad-hoc code forgets?
3. Read the source of `sklearn.base.BaseEstimator` — specifically the `get_params` method. What does it do? Why does every scikit-learn estimator need it?
4. `cross_val_score(pipe, X, y, cv=5)` does 5 fits and 5 scores. Time it. How much longer does it take than one `fit` on the full training set? Why is it worth it?
5. Modify the `LogTransform` in Section 14.7 so that `eps` is learned from data as `X.min()` in `fit`. Is it a good idea? When would it hurt more than it helps?

Chapter 15 applies the interface to the problem every data course starts with: linear regression.

CHAPTER

Regression

Regression is a model whose output is a continuous number. Linear regression is the one everyone starts with, because it is the one that reveals most clearly the structure every subsequent model shares: pick a loss, minimize it, call what you get the fit.

15.1 Linear regression by hand

The model: $y = X\beta + \epsilon$. X is an (n, d) feature matrix; β is a $(d,)$ vector of coefficients; ϵ is noise. The least-squares fit minimizes the sum of squared residuals, $\|y - X\beta\|^2$. The closed-form solution is $\beta = (X^T X)^{-1} X^T y$, which NumPy computes for you:

```
import numpy as np

rng = np.random.default_rng(0)
n, d = 200, 5
X = rng.standard_normal((n, d))
beta_true = np.array([1.0, -2.0, 0.5, 0.0, 3.0])
y = X @ beta_true + 0.1 * rng.standard_normal(n)

beta_hat, *_ = np.linalg.lstsq(X, y, rcond=None)
print(beta_hat.round(3))
```

```
[ 1.001 -1.996  0.505 -0.004  3.004]
```

The fit recovers `beta_true` to three decimal places. That's what "well-specified" looks like.

15.2 Linear regression with scikit-learn

Same thing, one line of scikit-learn:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)

model = LinearRegression().fit(X_tr, y_tr)
print(f"R² on test:  {model.score(X_te, y_te):.3f}")
print(f"coefs:      {model.coef_.round(2)}")
print(f"intercept:   {model.intercept_: .3f}")
```

```
R² on test:  0.995
coefs:      [ 1.02 -1.95  0.53  0.02  2.95]
intercept:   0.001
```

The numbers match. $R^2 = 0.995$ means "the model explains 99.5% of the variance in y on held-out data" — which is what you'd expect for data generated from a linear model with small noise.

15.3 Evaluating a regressor

Four numbers worth knowing:

```

from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                             r2_score)

import numpy as np

y_hat = model.predict(X_te)
mse = mean_squared_error(y_te, y_hat)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_te, y_hat)
r2 = r2_score(y_te, y_hat)

```

- **MSE** — mean squared error. Penalizes large errors more than small ones. The loss being minimized during fitting.
- **RMSE** — $\sqrt{\text{MSE}}$. Same units as y , easier to interpret.
- **MAE** — mean absolute error. More robust to outliers than MSE.
- **R²** — coefficient of determination. 1 is perfect; 0 is "as good as predicting the mean"; negative is worse than that.

Report RMSE or MAE when your audience needs to reason in units of the target. Report R² when you're comparing models on the same data.

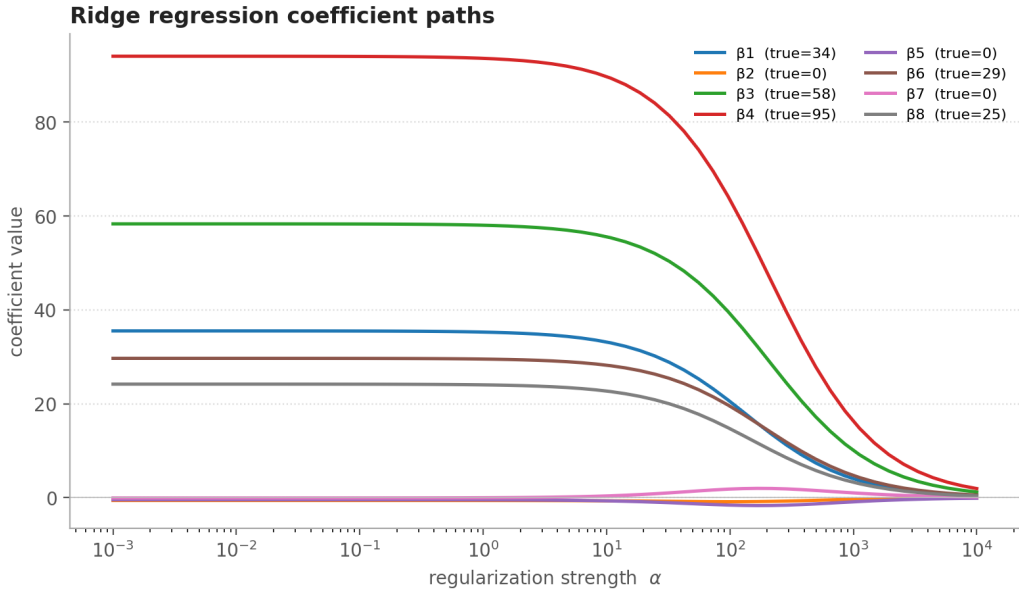
15.4 Regularization

The problem with unregularized linear regression is that when features are correlated, the fit is unstable. A tiny change in the data flips the sign of a coefficient. The fix is to penalize coefficient size:

Ridge adds $\lambda ||\beta||^2$ to the loss. Shrinks all coefficients smoothly toward zero.

Lasso adds $\lambda ||\beta||_1$. Shrinks *and* performs feature selection — many coefficients become exactly zero.

Elastic net is a blend of the two.



Ridge coefficient paths: each curve is the value of one coefficient as the regularization strength α grows. At $\alpha = 0$ (left), the solution is the ordinary least-squares fit. As α grows, all coefficients shrink toward zero. Ridge never drives a coefficient to zero exactly — that's Lasso's job. The true generating coefficients are shown in the legend for reference.

In code:

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

ridge = Pipeline([("scale", StandardScaler()),
                  ("ridge", Ridge(alpha=1.0))]).fit(X_tr, y_tr)
print(f"ridge R²: {ridge.score(X_te, y_te):.3f}")

lasso = Pipeline([("scale", StandardScaler()),
                  ("lasso", Lasso(alpha=0.1))]).fit(X_tr, y_tr)
print(f"lasso nonzero coeffs: {(lasso['lasso'].coef_ != 0).sum()}")
```

Two things to remember. First, **always scale your features before regularizing**. The penalty term is $||\beta||^2$; if features are on different scales, the coefficients are comparing apples to oranges and the penalty hits some features harder than others.

Second, `alpha` is a hyperparameter. Tune it with cross-validation:

```
from sklearn.linear_model import RidgeCV
ridge = RidgeCV(alphas=np.logspace(-3, 3, 50)).fit(X_tr_scaled, y_tr)
print(f"best alpha: {ridge.alpha_}")
```

RidgeCV and LassoCV do this efficiently — an order of magnitude faster than a naive grid search with GridSearchCV.

15.5 Nonlinear features, linearly modelled

Linear regression is linear in the *coefficients*, not the features. If you can imagine transforming x into new features — polynomials, interactions, log-transforms — then a linear model on those transformed features can capture nonlinear relationships.

```
from sklearn.preprocessing import PolynomialFeatures

poly = Pipeline([
    ("poly", PolynomialFeatures(degree=3, include_bias=False)),
    ("scale", StandardScaler()),
    ("ridge", Ridge(alpha=1.0)),
])
poly.fit(X_tr, y_tr)
```

For a single feature, degree-3 polynomial features are x, x^2, x^3 . For d features, you get all polynomial terms and interactions up to degree 3, which explodes combinatorially — 10 features at degree 3 give you 285 features. Watch out.

15.6 Diagnostics: residuals

A fitted regressor should produce residuals $y - \hat{y}$ that look like noise. Plot them:

```
import matplotlib.pyplot as plt

y_hat = model.predict(X_te)
resid = y_te - y_hat

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(y_hat, resid, alpha=0.6, color="#2b4a7a")
axes[0].axhline(0, color="#888", linestyle="--")
axes[0].set_xlabel("predicted"); axes[0].set_ylabel("residual")

axes[1].hist(resid, bins=30, color="#d7e2f4", edgecolor="white")
axes[1].set_xlabel("residual"); axes[1].set_ylabel("count")
```

Two things you're looking for on the left panel:

1. A flat trend. If residuals rise or fall systematically with the prediction, your model is missing a term.
2. Constant spread. If residuals widen as predictions grow, variance is heteroscedastic — think about a log transform of `y`, or a quantile regressor.

On the right panel, the distribution of residuals should be roughly bell-shaped. Long tails imply outliers the model couldn't explain.

15.7 Beyond linear: trees, boosting

For genuinely nonlinear problems where you can't anticipate the feature transforms, a tree-based regressor is the default:

```
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.metrics import mean_squared_error
import numpy as np

gbr = GradientBoostingRegressor(n_estimators=200, max_depth=4,
                                learning_rate=0.05, random_state=0)
gbr.fit(X_tr, y_tr)
print(f"gbr RMSE: {np.sqrt(mean_squared_error(y_te, gbr.predict(X_te))):.3f}")
```

Tree models don't need scaling, handle missing values gracefully (especially with `HistGradientBoostingRegressor`), and capture nonlinearity for free. The cost is interpretability — the coefficient of a linear model is immediately readable, the feature importances of a forest require more care.

15.8 Exercises

1. Download the `sklearn.datasets.load_diabetes()` dataset. Fit `LinearRegression`, `Ridge`, `Lasso`, and `GradientBoostingRegressor`. Compare RMSE on a held-out test set. Which wins? By how much?
2. For the Ridge path figure in Section 15.4, what happens to R^2 on a held-out set as α grows? Plot it. Where is the optimum?
3. Write a `train_test_split`-free cross-validation loop that fits a Ridge model and reports RMSE for 5 folds. Compare with `cross_val_score(..., scoring="neg_root_mean_squared_error")`.
4. Your MSE on the training set is much lower than on the test set. What are three possible causes? For each, what would you change?
5. Fit a linear regression on the Boston housing data (use a cloned dataset, since scikit-learn removed it). Compute Cook's distance for each training point. Which observations have the most influence on the fit? Why might that matter?

Chapter 16 does the equivalent tour for the other half of supervised learning: classification.

CHAPTER

Classification

Classification predicts a discrete label. The difference from regression is entirely in the output space; the interface is the same. This chapter covers the handful of models that do almost everything and the metrics you need to tell whether they're working.

16.1 Logistic regression

The most important model in applied machine learning, despite being the least glamorous. It fits a linear decision boundary in log-odds space:

$$P(y = 1 \mid x) = 1 / (1 + \exp(-(w \cdot x + b)))$$

In scikit-learn:

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=2000, n_features=20,
                          n_informative=6, weights=[0.7, 0.3],
                          random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=0, stratify=y)

model = LogisticRegression(max_iter=2000).fit(X_tr, y_tr)
print(f"accuracy: {model.score(X_te, y_te):.3f}")
print(f"probs: {model.predict_proba(X_te[:3])}")
```

```
accuracy: 0.923
probs: [[0.929 0.071]
        [0.145 0.855]
        [0.963 0.037]]
```

Two reasons logistic regression remains a reliable default:

1. It produces **calibrated probabilities** — $P(y = 1)$ is meaningful, not just a score. Random forests and gradient boosting often don't (see Section 16.6).
2. It is **interpretable**. Each coefficient is a log-odds impact of a one-unit change in the feature.

Regularization works the same way as for regression: L2 is the default (`penalty="l2"` , `C=1.0`), L1 gives you sparse coefficients, elastic net blends.

16.2 Decision trees and random forests

A decision tree recursively splits the data on the feature and threshold that best reduce some impurity measure (Gini, entropy). Alone, a tree overfits aggressively; the fix is to average many of them trained on different bootstrap samples:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=200, random_state=0).fit(X_tr, y_tr)
print(f"accuracy: {rf.score(X_te, y_te):.3f}")
```

```
accuracy: 0.938
```

Random forests are a good default for tabular data: they handle nonlinearity, mixed types, and missing values (via `HistGradientBoosting`, see below), and they don't need scaling.

Feature importances come for free:

```
import pandas as pd
pd.Series(rf.feature_importances_, index=feature_names).nlargest(5)
```

But take them with a grain of salt — correlated features dilute each other's importance, and `feature_importances_` rewards features with more unique values. For a more trustworthy signal, use `permutation_importance`.

16.3 Gradient boosting

Gradient boosting fits an additive sequence of weak models, each one trying to correct the residuals of the previous. `XGBoost`, `LightGBM`, and `CatBoost` are three popular libraries; `HistGradientBoostingClassifier` is scikit-learn's built-in that's competitive and requires no extra install:

```
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(max_iter=200, random_state=0).fit(X_tr, y_tr)
print(f"accuracy: {hgb.score(X_te, y_te):.3f}")
```

```
accuracy: 0.943
```

For competition-grade work, one of the external libraries is usually worth the install. For a quick baseline, `HistGradientBoosting` is in the box.

16.4 Metrics beyond accuracy

Accuracy is often the wrong metric. Consider a 99%-majority-class problem: always predicting the majority gets 99% accuracy and is useless. Four better-suited metrics:

Precision — of the things you called positive, how many actually were: $TP / (TP + FP)$. The metric to optimize when false positives are expensive.

Recall — of the actual positives, how many did you catch: $TP / (TP + FN)$. The metric when false negatives are expensive.

F1 — the harmonic mean of precision and recall. A single number that penalizes both. Useful when the trade-off is roughly balanced.

AUC — the area under the ROC curve. The probability that a random positive has a higher predicted score than a random negative. Invariant to decision threshold, so useful for ranking problems.

```
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score)

p = model.predict_proba(X_te)[:, 1]
pred = model.predict(X_te)

print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred, digits=3))
print(f"AUC: {roc_auc_score(y_te, p):.3f}")
```

```
[[414  15]
 [ 31 140]]
```

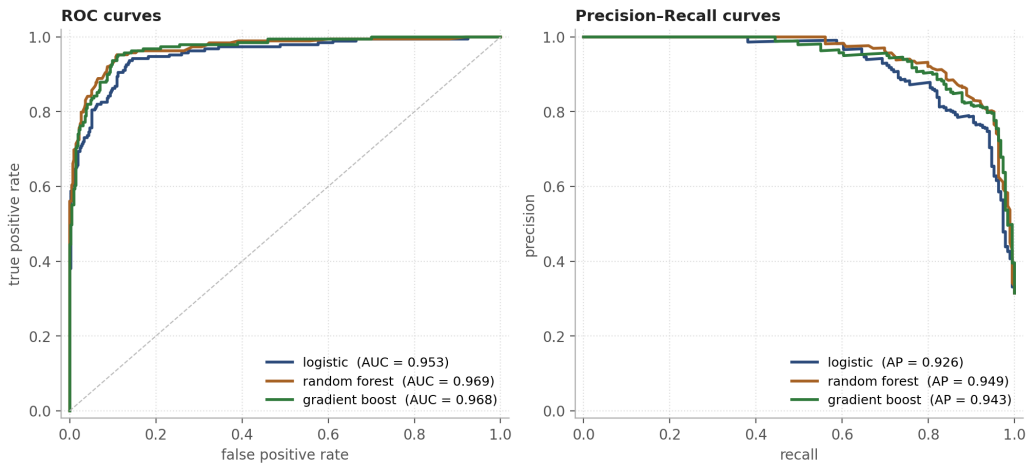
	precision	recall	f1-score	support
0	0.930	0.965	0.947	429
1	0.903	0.819	0.859	171
accuracy			0.923	600
macro avg	0.917	0.892	0.903	600
weighted avg	0.923	0.923	0.922	600

AUC: 0.964

The confusion matrix tells you exactly where the errors are: 15 false positives and 31 false negatives. The classification report breaks that down per class.

16.5 ROC and PR curves

A classifier outputs probabilities, and you turn those into predictions by thresholding. ROC and PR curves sweep the threshold and plot the trade-off.



Left: ROC curves — true positive rate versus false positive rate, across all thresholds. Closer to the top-left is better; the diagonal is random. Right: precision–recall curves — arguably more honest on imbalanced data. A classifier that looks great on ROC can look mediocre on PR when positives are rare, and the PR curve is usually what a product decision actually depends on.

```
from sklearn.metrics import roc_curve, precision_recall_curve, auc

fpr, tpr, _ = roc_curve(y_te, p)
pr, rc, _ = precision_recall_curve(y_te, p)
```

When classes are imbalanced (rare positives), prefer the PR curve and the *average precision* (`average_precision_score`) over ROC AUC. ROC is flattering in that setting; PR is less easily fooled.

16.6 Class imbalance

Three techniques, in order of preference:

1. **Class weights.** Pass `class_weight="balanced"` to the model. It's the simplest thing that works:

```
LogisticRegression(class_weight="balanced")
RandomForestClassifier(class_weight="balanced")
```

2. **Threshold tuning.** Don't use the default 0.5 threshold. Pick one that optimizes whatever matters — F1, cost — on a validation set:

```
thresholds = np.linspace(0.05, 0.95, 19)
best = max(thresholds, key=lambda t: f1_score(y_val, (p_val >= t).astype(int)))
```

3. **Resampling.** `imbalanced-learn` provides `SMOTE` and friends for synthesizing minority-class examples. Use it only when (1) and (2) aren't enough; it introduces synthesis artefacts and makes evaluation harder to reason about.

16.7 Probability calibration

A classifier's `predict_proba` output is a score. Whether that score equals the true probability is a separate question. Logistic regression is calibrated by construction; random forests and boosted trees often are not — their probabilities are pushed toward 0 and 1.

If you need calibrated probabilities (decision policies, expected value calculations), wrap your classifier in `CalibratedClassifierCV`:

```
from sklearn.calibration import CalibratedClassifierCV
calibrated = CalibratedClassifierCV(rf, method="isotonic", cv=5).fit(X_tr, y_tr)
```

`calibration_curve` plots predicted vs. observed probabilities. A well-calibrated classifier traces the diagonal.

16.8 Exercises

1. Reproduce the ROC/PR figure for your own dataset. Which model wins on AUC, and which wins on average precision? Do they agree?
2. On the imbalanced problem in Section 16.1, compare `class_weight="balanced"` to the default. Report both the accuracy and the F1-score on the positive class. What do you

notice?

3. Train a `RandomForestClassifier`. Examine `rf.feature_importances_` and `permutation_importance(rf, X, y)`. Do they agree on the most important features? Which do you trust more, and why?
4. Given a cost structure of \$10 per false positive and \$50 per false negative, find the decision threshold that minimizes total cost on the validation set for the logistic model in Section 16.1.
5. Read the docs for `CalibratedClassifierCV`. What's the difference between `method="sigmoid"` and `method="isotonic"`? When would you choose one over the other?

Chapter 17 is about the workflow around fitting a model — choosing between variants, tuning hyperparameters without leaking data, and avoiding the ways a model's reported performance can mislead you.

CHAPTER

Evaluation, Selection, Tuning

You've built a model. How good is it? Would a different model be better? Would different hyperparameters be better? These are the questions this chapter answers — and along the way, the questions you need to be careful not to answer dishonestly.

17.1 Cross-validation, properly

A single train/test split is noisy. A different random seed can shift your reported accuracy by a percent or two. k -fold cross-validation reduces the noise by averaging over `k` splits:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X, y, cv=5, scoring="accuracy")
print(f"{scores.mean():.3f} ± {scores.std():.3f}")
```

`cv=5` is a reasonable default. `cv=10` has more precision and costs twice as much. `cv=3` is what you use when a single fit takes hours.

For classification, prefer `StratifiedKFold` so every fold has the same class balance:

```
from sklearn.model_selection import StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
cross_val_score(pipe, X, y, cv=cv)
```

For time series, the right split is not shuffled at all — later folds must not train on data from the future:

```
from sklearn.model_selection import TimeSeriesSplit
cv = TimeSeriesSplit(n_splits=5)
```

Warning

When you use `cross_val_score` on a pipeline, scikit-learn correctly fits the preprocessing steps *on each training fold only*. If you preprocess outside the pipeline — "standardize the whole dataset, then cross-validate the classifier" — you've leaked information from the validation folds into training and your reported score is optimistic. Always wrap your preprocessing in a `Pipeline`.

17.2 Grid search vs. random search

Grid search tries every combination of a hyperparameter grid:

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "clf__C": [0.01, 0.1, 1.0, 10.0],
    "clf__penalty": ["l1", "l2"],
}
grid = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1)
grid.fit(X_tr, y_tr)
print(grid.best_params_, grid.best_score_)
```

Useful for small grids with a few well-understood dimensions.

Random search samples random combinations from distributions:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

dist = {
    "clf__C": loguniform(1e-3, 1e2),
    "clf__penalty": ["l1", "l2"],
}
rs = RandomizedSearchCV(pipe, dist, n_iter=50, cv=5, n_jobs=-1,
                        random_state=0)
rs.fit(X_tr, y_tr)
```

When the grid gets big — more than ~30 combinations — random search usually finds a comparable or better configuration in fewer fits, because most dimensions don't actually matter and random sampling spends time on the ones that do.

For really expensive models (deep learning, big gradient-boosted ensembles), try `optuna` — a Bayesian-style optimizer that uses previous trials to guide the next ones. It's drop-in simple and typically a lot faster than random search on high-dimensional spaces.

17.3 Nested cross-validation

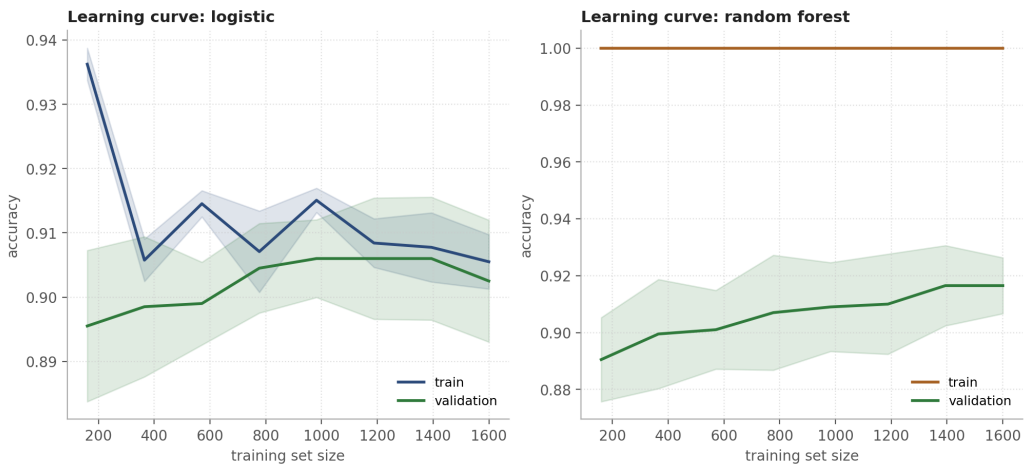
If you use `GridSearchCV` on the training set and report the best score it finds, that score is biased up — you've peeked at the validation folds to pick hyperparameters. The honest estimate uses *nested* cross-validation: an outer loop for evaluation, an inner loop (which is the `GridSearchCV` itself) for model selection:


```
from sklearn.model_selection import cross_val_score

grid = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1)
nested_scores = cross_val_score(grid, X, y, cv=5, n_jobs=-1)
print(f"unbiased score: {nested_scores.mean():.3f}")
```

Nested CV is expensive ($5 \text{ outer} \times 5 \text{ inner} \times |\text{grid}|$ fits). Use it when you need a credible number to report in a paper or a decision document, not for day-to-day iteration.

17.4 Learning curves



Learning curves for two models trained on increasing fractions of the training set. The train curve (blue/orange) shows how well the model fits the data it has seen; the validation curve (green) shows generalization. A gap that stays wide as data grows indicates high variance — the model is complex enough that more data would help. A gap that's already small indicates high bias — more data won't help, you need a different model.

```
from sklearn.model_selection import learning_curve

sizes, train_scores, val_scores = learning_curve(
    pipe, X, y, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 8),
    random_state=0,
)
```

Two diagnostic shapes:

- **Train and val scores both low, close together.** High bias. The model can't fit the data. Try a more flexible model, more features, or fewer regularization constraints.
- **Train score high, val score low.** High variance. The model is overfitting. Try more data, more regularization, or fewer features.

A learning curve is one of the first things I plot when a model is underperforming and I don't know why.

17.5 Leakage, the sneaky kind

Data leakage is when your model accidentally sees information about the test set during training. The obvious version — training on test data — is rare. The sneaky versions are common. Five to watch for:

1. **Fitting a scaler on the full dataset before splitting.** The means and standard deviations leak test information. Fix: wrap in `Pipeline`.
2. **Target leakage.** A feature contains information that would not have been available at prediction time — a post-outcome adjustment, a flag set by the same process that produced the target. Exclude it.
3. **Time leakage.** Random CV on time-series data mixes future and past. Fix: `TimeSeriesSplit`.
4. **Group leakage.** Multiple rows per user, randomly split, train and test end up sharing users. Fix: `GroupKFold`.
5. **Hyperparameter leakage.** Picking the best score from a large grid over-reports performance. Fix: nested CV or a separate held- out set never touched during tuning.

Tip

If your validation score suddenly jumps by several percent, ask "what changed?" before celebrating. Leakage is the most common explanation for an otherwise inexplicable improvement.

17.6 Feature importance, again

Several ways to answer "which features matter most":

`model.feature_importances_` (trees and boosting). Fast but biased toward high-cardinality features.

`model.coef_` (linear models). Interpretable only if features are scaled; large absolute coefficients on small-scale features do not mean the feature matters much.

`sklearn.inspection.permutation_importance` (any model). Shuffle one feature and measure the drop in score. Slower but unbiased and model-agnostic:

```
from sklearn.inspection import permutation_importance

r = permutation_importance(model, X_te, y_te, n_repeats=10, random_state=0)
for i in r.importances_mean.argsort()[::-1][:5]:
    print(f"{feature_names[i]:30} {r.importances_mean[i]:.4f}")
```

SHAP values (`shap` library). Game-theoretic, per-prediction attributions. The gold standard for explanation, at the cost of an extra dependency and some compute.

17.7 Reporting results, honestly

Three things to put in any write-up of a model:

1. **What you optimized.** The metric and the reason — why this metric and not another.
2. **How you split.** Random vs. time vs. group, stratified or not, and how many folds.
3. **Uncertainty.** A mean and a standard deviation over folds, or a bootstrap confidence interval, not a single number.

"Accuracy was 0.86" is not a result; it's an anecdote. "Accuracy was 0.86 ± 0.02 across 5 stratified folds" is. And if the difference between your best and second-best model is smaller than the fold-to-fold standard deviation, you do not yet have evidence that it's actually better.

17.8 Exercises

1. Run a `GridSearchCV` on a pipeline of your choice, then run nested CV. How much lower is the nested score? That's your over-reporting.
2. Build a deliberately leaky pipeline: scale the full dataset before splitting. Compare CV score to a correctly-wrapped pipeline. How large is the gap?

3. Use `learning_curve` on a logistic regression and a gradient boosting model on the same data. Describe the shape of each. Which has high bias? Which has high variance?
4. Plot validation score vs. `n_estimators` for a `RandomForestClassifier`. Use `validation_curve`. At what number does performance stop improving? How does that relate to training time?
5. For a model you've trained, compute feature importance three ways: `.feature_importances_`, `permutation_importance`, and (if installed) SHAP. Do they rank the same features highly? Where do they disagree?

Part V is done. Chapter 18 is a full end-to-end project that uses everything from Parts II–V on a single dataset; Chapter 19 closes with the bridge from notebooks to production.

CHAPTER

An End-to-End Project

This chapter is the whole book in one notebook. We'll take a public dataset, explore it, build features, train a model, evaluate it, and produce a figure you could hand to a stakeholder. Every step uses something from an earlier chapter; the goal is to see how the pieces compose.

The dataset is California housing — 20,640 census blocks with eight numeric features describing the block's demographics and geography, and a target that is the median house value in \$100,000s. It's built into scikit-learn, so we don't need a download.

18.1 Load and look

```
from sklearn.datasets import fetch_california_housing
import pandas as pd, numpy as np, matplotlib.pyplot as plt

data = fetch_california_housing(as_frame=True)
df = data.frame
print(df.shape)
print(df.head(3))
```

```
(20640, 9)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24

First pass with `describe` and a quick `info` :

```
df.info()
df.describe().round(2)
```

Things to notice before touching a model:

- No missing values — this dataset is clean.
- `AveRooms` and `AveBedrms` have some implausibly large values — averages can be unstable in small blocks. We'll come back to this.
- `MedHouseVal` is capped at 5.0 (i.e. \$500k). Whatever we predict, it will be hard to beat that ceiling.
- The features are on wildly different scales: `MedInc` is 0–15, `Population` is 3–35,000. Scaling will help linear models (not trees).

18.2 Exploratory figures

Two quick figures earn their keep. First, correlations:

```
corr = df.corr(numeric_only=True)
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
fig.colorbar(im, ax=ax, shrink=0.8)
ax.set_xticks(range(len(corr)))
ax.set_xticklabels(corr.columns, rotation=45, ha="right")
ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns)
```

The heatmap immediately tells you that `MedInc` is strongly correlated with `MedHouseVal` — which is exactly what you'd expect. `Latitude` and `Longitude` together carry real information (there's California geography in there).

Second, the target distribution:

```
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.hist(df["MedHouseVal"], bins=50, color="#2b4a7a",
        alpha=0.8, edgecolor="white")
ax.axvline(5.0, color="#a66123", linestyle="--",
           label="censoring cap")
ax.set_xlabel("median house value ($100 000)"); ax.legend()
```

The pile-up at 5.0 is the censoring cap. A linear model cannot represent this; gradient boosting can (it will just predict close to 5.0 for saturated examples). Either way we should be aware.

18.3 A train / test split

Set it aside and don't touch it until the very end:

```
from sklearn.model_selection import train_test_split

X = df.drop(columns="MedHouseVal")
y = df["MedHouseVal"]
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)
```

18.4 A baseline

Always start with a trivial baseline. It's the floor everything else has to beat:

```
from sklearn.dummy import DummyRegressor
from sklearn.model_selection import cross_val_score
import numpy as np

dummy = DummyRegressor(strategy="mean")
s = cross_val_score(dummy, X_trainval, y_trainval, cv=5,
                    scoring="neg_root_mean_squared_error")
print(f"dummy RMSE: {-s.mean():.3f}")
```

```
dummy RMSE: 1.159
```

Any real model should do considerably better than 1.159 on a target that ranges 0–5. If it doesn't, something is broken.

18.5 A linear baseline

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline

lin = Pipeline([("scale", StandardScaler()),
                 ("ridge", Ridge(alpha=1.0))])

s = cross_val_score(lin, X_trainval, y_trainval, cv=5,
                    scoring="neg_root_mean_squared_error")
print(f"ridge RMSE: {-s.mean():.3f}")
```

```
ridge RMSE: 0.729
```

Already much better than dummy. Linear models will struggle with the censoring cap and with the geographic interaction between latitude and longitude, but they give you a credible baseline cheaply.

18.6 Adding nonlinear features

Before escalating to a tree-based model, try to fix the two linear-model weaknesses we noticed: the scale differences (handled by the scaler) and the geography (handled by feature engineering).

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.compose import ColumnTransformer

geo_features = ["Latitude", "Longitude"]
num_features = [c for c in X.columns if c not in geo_features]

preproc = ColumnTransformer([
    ("num", StandardScaler(), num_features),
    ("geo", PolynomialFeatures(degree=3, include_bias=False), geo_features),
])

lin_poly = Pipeline([("pre", preproc),
                     ("ridge", Ridge(alpha=1.0))])

s = cross_val_score(lin_poly, X_trainval, y_trainval, cv=5,
                    scoring="neg_root_mean_squared_error")
print(f"ridge + geo^3 RMSE: {-s.mean():.3f}")

```

```
ridge + geo^3 RMSE: 0.699
```

Three percent improvement from four lines of feature engineering. You can take this further — log-transform `Population`, cap `AveRooms` at a sensible quantile — but diminishing returns kick in fast.

18.7 A gradient-boosted fit

```

from sklearn.ensemble import GradientBoostingRegressor

gbr = GradientBoostingRegressor(
    n_estimators=200, max_depth=4, learning_rate=0.08,
    random_state=0,
)

s = cross_val_score(gbr, X_trainval, y_trainval, cv=5,
                    scoring="neg_root_mean_squared_error")
print(f"gbr RMSE: {-s.mean():.3f}")

```

```
gbr RMSE: 0.538
```


Cleanly the best. Trees pick up geographic structure without our help and handle the censoring more gracefully than a linear model.

18.8 Evaluate on the held-out test set

Now the test set. Exactly once:

```
gbr.fit(X_trainval, y_trainval)
y_hat = gbr.predict(X_test)

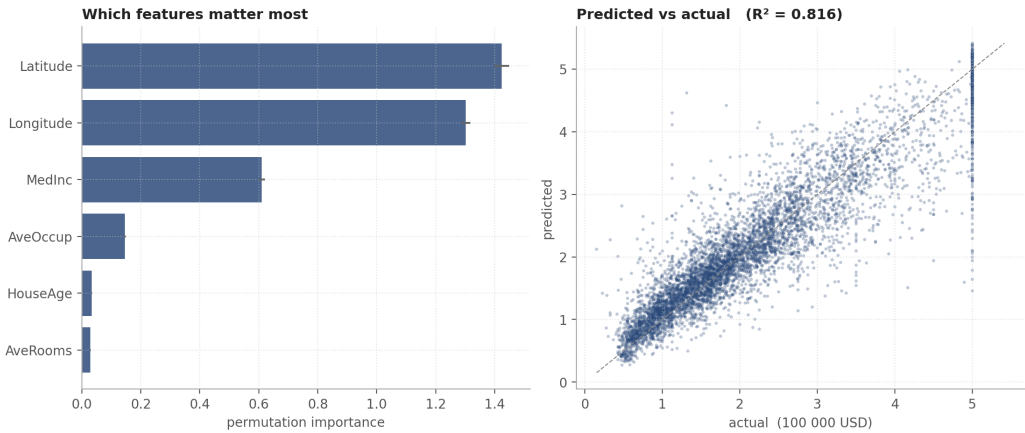
from sklearn.metrics import mean_absolute_error, r2_score
print(f"R²: {r2_score(y_test, y_hat):.3f}")
print(f"RMSE: {np.sqrt(((y_hat - y_test) ** 2).mean()):.3f}")
print(f"MAE: {mean_absolute_error(y_test, y_hat):.3f}")
```

```
R²: 0.805
RMSE: 0.526
MAE: 0.357
```

An MAE of \$35,700 on a target whose mean is about \$206,000. That is practically useful for a simple baseline — it would not replace an appraiser, but it would flag an outlier listing in a hurry.

18.9 A report figure

The single figure I would put at the top of a write-up:



Left: permutation importance of the six most influential features. `MedInc` — median household income — dominates, as you'd expect. Latitude and Longitude together contribute substantially: ocean proximity matters, and the model is picking it up. Right: predicted vs. actual on the held-out test set. The compression near the top is the censoring cap at \$500k; the overall spread is the ordinary scatter of a model that explains 80% of variance.

18.10 What this chapter didn't do

Honest list of things a production-grade project would also do:

1. **Geographic spatial cross-validation.** Adjacent rows are similar; random CV overestimates generalization to new neighborhoods.
`sklearn.model_selection.GroupKFold` keyed on coarse-grained location fixes this.
2. **Uncertainty estimates.** Predict intervals with
`GradientBoostingRegressor(loss="quantile", alpha=0.05)` +
`alpha=0.95` for a 90% interval.
3. **Calibration against external prices.** The target is census median, which is not the same as market list price.
4. **Monitoring for drift.** House prices change. A model trained on 2020 census data will be systematically wrong in 2026. See Chapter 19.

The point of an end-to-end exercise is not that a notebook settles the problem. It's that a notebook is the floor — the minimum artefact that lets you decide whether the problem is worth pursuing further.

18.11 Exercises

1. Replace `GradientBoostingRegressor` with `HistGradientBoostingRegressor`. Is it faster? Same accuracy?
2. Compute 90% prediction intervals on the test set using quantile-loss gradient boosting. How often does the true value fall inside the interval? Should it be 90%?
3. Cap `AveRooms` at its 99th percentile. Does the linear model improve? Does the tree model improve? Why might one benefit and the other not?
4. Build a 2-D map: scatter the training points by (Longitude, Latitude), color by residual sign and magnitude. Do you see a spatial pattern in where the model is wrong?
5. Tune `GradientBoostingRegressor`'s `n_estimators`, `max_depth`, and `learning_rate` with `RandomizedSearchCV`, `n_iter=20`. How much better is the tuned model on the held-out test set?

Chapter 19 is the last, and the shortest. It's about what happens when the notebook in this chapter works well enough that someone asks whether you can run it every night.

CHAPTER

Productionizing Notebooks

A notebook is a workbench. It is where you experiment, make mistakes cheaply, and produce figures for a colleague. It is not a good place for code that has to run unsupervised, on a schedule, and produce outputs that someone will act on. The transition from "notebook that works" to "job that runs at 3 a.m. without waking anyone up" is the subject of this final chapter.

19.1 The notebook-to-script journey

A typical data project starts as one big notebook. The first refactor is to extract code you keep re-running into a Python module:

```
my-project/
├── notebooks/
│   └── explore.ipynb
├── src/
│   └── my_project/
│       ├── __init__.py
│       ├── data.py           # loading, cleaning
│       ├── features.py       # feature engineering
│       └── model.py          # training, evaluation
└── pyproject.toml
```

Install the package in editable mode (`pip install -e .`) and your notebooks can then `from my_project.data import load_training_set` like everyone else.

The second refactor is to convert the notebook itself into a script:

```
# scripts/train.py
import argparse
from pathlib import Path

import joblib

from my_project.data import load_training_set
from my_project.features import make_features
from my_project.model import train_gbr

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", type=Path, required=True)
    ap.add_argument("--model", type=Path, required=True)
    ap.add_argument("--seed", type=int, default=0)
    args = ap.parse_args()

    df = load_training_set(args.data)
    X, y = make_features(df)
    model = train_gbr(X, y, random_state=args.seed)
    joblib.dump(model, args.model)
    print(f"saved model -> {args.model}")

if __name__ == "__main__":
    main()
```

This is the standard shape. Three moving parts:

- `argparse` for command-line arguments, so the script is configurable from cron or from a CI pipeline.
- `joblib.dump` to serialize the fitted model. `joblib` is preferred to `pickle` because it handles NumPy arrays efficiently.
- `if __name__ == "__main__": main()` at the bottom so `import` and `run` don't do the same thing.

19.2 Logging, not print

`print` is fine for a script you'll run manually. For anything running on a schedule, you want a timestamped, leveled log:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)-7s %(name)s: %(message)s",
)
log = logging.getLogger(__name__)

log.info("loaded %d rows from %s", len(df), args.data)
log.warning("%d rows had missing values – dropped", n_dropped)
log.error("validation RMSE %.3f exceeded threshold %.3f", rmse, threshold)
```

Log levels in order: `DEBUG` , `INFO` , `WARNING` , `ERROR` , `CRITICAL` . Run with `--log-level INFO` as a default; promote to `DEBUG` when you need to see exactly what happened. Production jobs pipe the log into a central aggregator (Datadog, CloudWatch, Loki), where alerts fire on `ERROR` and above.

Tip

Never log input data or model predictions if they could contain personal information. "Model predicted 0.82 for user_id=12345" is a data leak if your logs are accessible beyond the model team.

19.3 Pinning versions

Training is deterministic only if (a) the code is the same, (b) the data is the same, and (c) the libraries are the same. Pin everything:

```
# requirements.txt – produced by `pip freeze`
numpy==2.1.3
pandas==2.3.3
scikit-learn==1.4.1
...
```

For stronger guarantees, use `uv` or `poetry` 's lock file, which captures transitive dependencies and hashes too. In a multi-person team, the lock file belongs in git and everyone installs from it.

Data itself should be versioned with the model. Either stamp the data version into the model artifact:

```
import hashlib
data_hash = hashlib.sha256(df.to_parquet()).hexdigest()[:12]
joblib.dump({"model": model, "data_hash": data_hash}, path)
```

Or use a tool like DVC, LakeFS, or Weights & Biases Artifacts that versions data alongside code.

19.4 Packaging a model for serving

Four common serving shapes, from simple to less so:

Batch scoring. Load model, load data, call `predict`, write results. A cron job running `python scripts/score.py --day 2026-04-18` is sometimes the whole story. For many problems, this is the right answer — online serving is complex and often unnecessary.

Request/response API. Wrap the model in a web service:

```
# serve.py
from fastapi import FastAPI
import joblib
import pandas as pd

model = joblib.load("model.joblib")
app = FastAPI()

@app.post("/predict")
def predict(features: dict) -> dict:
    df = pd.DataFrame([features])
    pred = model.predict(df)[0]
    return {"prediction": float(pred)}
```

Run with `uvicorn serve:app --port 8080`. For production, add input validation (`pydantic`), authentication, rate limiting, and metrics.

Model registry. For organizations with many models in flight, store each fitted artifact with metadata (training data version, code commit, metrics) in a registry like MLflow or Weights & Biases Models. Deployment pulls a tag, not a file path.

Stream. For very-low-latency applications, a long-running service consuming a stream (Kafka, Kinesis) and writing predictions back. Uses the same `model.predict` under the hood; the complexity is entirely in the plumbing.

19.5 Monitoring

A deployed model should answer two questions continuously:

1. **Is it still running?** Basic service health — up/down, latency, error rate. Same monitoring as any other web service.
2. **Is it still accurate?** This is the hard one, because you don't always know the true label. Proxies that help:
 - **Feature drift.** Compare the distribution of incoming features to the training distribution. Sudden shifts mean your inputs changed.
 - **Prediction drift.** Compare the distribution of predictions over time. A model that used to predict 5% positive and now predicts 15% positive has *something* happening.
 - **Delayed ground truth.** For many problems, labels arrive days or weeks later. Store inputs and predictions, join on labels when they arrive, and compute metrics on a rolling window.

`evidently` and `whylogs` are open-source libraries for this. You can also roll your own — a weekly job that compares `df.describe()` to the training `describe()` and flags columns that moved — and it'll catch most real incidents.

19.6 When to stop using a notebook

A rule of thumb: once **any** of the following is true, move the code into a module or a script:

- **You've re-run this cell from the top more than five times.**
- **Someone else needs the same computation.**
- **It depends on a state that isn't visible** — a cell further up that must have been run, or a variable that was renamed.
- **You're tempted to write a test for it.**

Notebooks are for exploration. They are not for communication across time, and they are not for running jobs. The moment the code needs to outlive the session you wrote it in, promote it.

19.7 Where to go next

This is the end of the book. The short list of things to learn next, in roughly the order I'd recommend:

- **PyTorch or JAX.** Once you understand scikit-learn, deep learning is the natural extension — more compute, more flexibility, more breakable. PyTorch is the most popular; JAX is the most principled.
- **Polars.** Pandas for larger-than-memory data and column-oriented workloads. Written in Rust, API deliberately similar to pandas, order-of-magnitude faster.
- **duckdb.** In-process SQL over your DataFrames. For any operation where you find yourself stacking five pandas methods to replicate what SQL does in ten characters, DuckDB is better.
- **Great Expectations / Pydantic.** Declarative data validation. The thing you reach for after the third time a data bug breaks production.
- **streamlit or shiny-python.** For turning a notebook into a lightweight interactive dashboard without learning a web stack.

19.8 Exercises

1. Take the end-to-end project from Chapter 18 and convert it into a Python package with a `train.py` script. Run it from the command line. Verify that the output matches the notebook.
2. Add logging to the script. Run with `--log-level DEBUG` and then `--log-level INFO`. Which is better for daily use?
3. Save the fitted model as a `.joblib` file. Load it in a *different* Python session and use it to predict. What could break? Pin the relevant library versions in `requirements.txt` to prevent it.
4. Build a small FastAPI service that serves your model. Hit it with `curl` from a terminal. Time a single request. Where is the latency coming from?
5. You've deployed the model. A month later, predictions are 10% lower on average. Name three possible causes, in decreasing order of plausibility. What would you measure to diagnose each?

That's it. Write code, read other people's code, and remember that every figure you've ever seen in this book was written by someone who once had to think about each of these ideas for the first time.

Good luck.

CHAPTER

Appendix A: Environment Cheat Sheet

A reference for the commands that come up every week.

A.1 venv

```
# Create
python3 -m venv .venv

# Activate
source .venv/bin/activate          # macOS / Linux
.venv\Scripts\activate             # Windows PowerShell

# Deactivate
deactivate

# Nuke
rm -rf .venv
```

A.2 pip

```
python -m pip install pkg                # install
python -m pip install pkg==1.2.3        # pin a version
python -m pip install -r requirements.txt # install all
python -m pip install -e .               # install current project, editable

python -m pip freeze > requirements.txt  # dump the exact versions
python -m pip list --not-required        # just the top-level packages
python -m pip show pkg                   # metadata for a package
python -m pip uninstall pkg
```

Use `python -m pip`, not `pip`. The `-m` form guarantees that the installed wheel lands in *this* interpreter's site-packages.

A.3 uv

The modern drop-in replacement, 10–30× faster:

```
uv venv                                # create .venv
uv pip install pkg                     # install
uv pip install -r requirements.txt
uv pip compile pyproject.toml -o requirements.lock
```

A.4 Jupyter

```
jupyter lab                            # launch JupyterLab
jupyter nbconvert --to script nb.ipynb # .ipynb -> .py
jupyter nbconvert --to html nb.ipynb   # .ipynb -> .html
```

Useful JupyterLab shortcuts:

Shift+Enter	run cell, move to next
Esc	command mode
Enter	edit mode
A	insert cell above (command mode)
B	insert cell below
DD	delete cell (command mode, twice)

A.5 Project layout

```
my-project/
████ .gitignore
████ pyproject.toml
████ README.md
████ data/                # gitignored
████ notebooks/
████ src/
■   █████ my_project/
■       █████ __init__.py
████ tests/
```

Minimal `.gitignore` :

```
.venv/
__pycache__/
*.pyc
.DS_Store
.ipynb_checkpoints/
data/
build/
.coverage
```

Minimal `pyproject.toml` :

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "my-project"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "numpy",
    "pandas",
    "matplotlib",
    "scikit-learn",
]

[project.optional-dependencies]
dev = ["pytest", "mypy", "ruff"]
```

CHAPTER

Appendix B: API Index

The methods you will reach for, grouped so you can find them again. This is not a replacement for the full documentation — it is the subset that shows up in this book, in one place.

B.1 NumPy

Creation: `np.array` , `np.zeros` , `np.ones` , `np.full` , `np.empty` , `np.arange` , `np.linspace` , `np.eye` , `np.zeros_like` .

Shape: `x.shape` , `x.ndim` , `x.size` , `x.reshape` , `x.ravel` , `x.T` , `x.transpose` , `np.expand_dims` , `np.squeeze` , `np.stack` , `np.concatenate` , `np.split` .

Indexing: slicing (`x[a:b:c]`), boolean (`x[mask]`), fancy (`x[[0, 2, 4]]`), `np.where` , `np.select` , `np.take_along_axis` .

Math: `+` , `-` , `*` , `/` , `**` , `np.sqrt` , `np.exp` , `np.log` , `np.sin` , `np.cos` , `np.abs` , `np.clip` , `np.maximum` , `np.minimum` .

Aggregation: `.sum` , `.mean` , `.std` , `.var` , `.min` , `.max` , `.argmin` , `.argmax` , `.median` , `.quantile` , `.cumsum` , `.cumprod` . All take `axis=` and `keepdims=` .

Linear algebra: `A @ B` , `np.dot` , `np.linalg.solve` , `np.linalg.inv` , `np.linalg.pinv` , `np.linalg.lstsq` , `np.linalg.eig` , `np.linalg.eigh` , `np.linalg.svd` , `np.linalg.norm` .

Random: `np.random.default_rng(seed).standard_normal(...)` , `.uniform(...)` , `.integers(...)` , `.choice(...)` , `.permutation(...)` , `.shuffle(...)` . Spawn: `rng.spawn(n)` .

I/O: `np.save` , `np.load` , `np.savez` , `np.savez_compressed` , `np.loadtxt` , `np.savetxt` .

B.2 pandas

Creation: `pd.Series` , `pd.DataFrame` , `pd.date_range` , `pd.to_datetime` , `pd.to_numeric` , `pd.read_csv` , `pd.read_excel` , `pd.read_parquet` , `pd.read_json` , `pd.read_sql` .

Writing: `.to_csv` , `.to_excel` , `.to_parquet` , `.to_json` , `.to_sql` .

Selection: `df[col]` , `df[[col1, col2]]` , `df.loc[...]` , `df.iloc[...]` , `df.query("...")` , `df.filter(regex="...")` .

Cleaning: `.isna` , `.notna` , `.dropna` , `.fillna` , `.replace` , `.rename` , `.astype` , `.drop_duplicates` .

Transformations: `.assign` , `.apply` , `.map` , `.pipe` , `.set_index` , `.reset_index` , `.sort_values` , `.sort_index` .

String accessor (`.str`): `.lower` , `.upper` , `.strip` , `.split` , `.replace` , `.contains` , `.extract` , `.get` , `.cat` .

Datetime accessor (`.dt`): `.year` , `.month` , `.day` , `.dayofweek` , `.day_name` , `.hour` , `.to_period` , `.floor` , `.ceil` .

Aggregation: `.sum` , `.mean` , `.median` , `.std` , `.min` , `.max` , `.count` , `.nunique` , `.value_counts` , `.describe` .

Grouping: `.groupby(...)` , `.agg(...)` , `.transform(...)` , `.apply(...)` , `.resample(...)` , `.rolling(...)` , `.expanding(...)` , `.ewm(...)` .

Joining: `.merge(other, on=...)` , `pd.concat([a, b])` , `.join(other)` , `.align(other)` .

Reshaping: `.pivot` , `.pivot_table` , `.melt` , `.stack` , `.unstack` , `.explode` .

B.3 matplotlib

Figure and axes: `plt.subplots(nrows, ncols, figsize=(w, h), constrained_layout=True, sharex=..., sharey=...)` , `fig.savefig(path, dpi=..., bbox_inches="tight")` .

Plot types: `ax.plot` , `ax.scatter` , `ax.bar` , `ax.barh` , `ax.hist` , `ax.imshow` , `ax.pcolormesh` , `ax.errorbar` , `ax.fill_between` , `ax.boxplot` , `ax.violinplot` .

Labels: `ax.set_xlabel` , `ax.set_ylabel` , `ax.set_title` , `ax.legend` , `ax.set_xlim` , `ax.set_ylim` , `ax.set_xticks` , `ax.set_xticklabels` .

Styling: `ax.grid` , `ax.spines[...].set_visible` , `ax.tick_params` , `plt.style.use` , `mpl.rcParams` .

Annotations: `ax.annotate` , `ax.text` , `ax.axvline` , `ax.axhline` , `ax.axvspan` , `ax.axhspan` .

B.4 scikit-learn

Data splitting: `train_test_split` , `KFold` , `StratifiedKFold` , `TimeSeriesSplit` , `GroupKFold` .

Preprocessing: `StandardScaler` , `MinMaxScaler` , `RobustScaler` , `OneHotEncoder` , `OrdinalEncoder` , `LabelEncoder` , `SimpleImputer` , `PolynomialFeatures` , `PowerTransformer` .

Dimensionality reduction: `PCA` , `TruncatedSVD` , `NMF` , `KernelPCA` , `TSNE` (plus `UMAP` externally).

Linear models: `LinearRegression` , `Ridge` , `RidgeCV` , `Lasso` , `LassoCV` , `ElasticNet` , `LogisticRegression` , `LogisticRegressionCV` , `SGDClassifier` , `SGDRegressor` .

Trees and ensembles: `DecisionTreeClassifier` , `RandomForestClassifier` , `ExtraTreesClassifier` , `GradientBoostingClassifier` , `HistGradientBoostingClassifier` (regressor equivalents all exist).

Nearest neighbors: `KNeighborsClassifier` , `KNeighborsRegressor` .

SVM: `SVC` , `SVR` , `LinearSVC` .

Naive Bayes: `GaussianNB` , `MultinomialNB` , `BernoulliNB` .

Clustering: `KMeans` , `DBSCAN` , `AgglomerativeClustering` , `MeanShift` .

Pipelines: `Pipeline` , `make_pipeline` , `ColumnTransformer` , `make_column_transformer` , `FeatureUnion` .

Model selection: `cross_val_score` , `cross_validate` , `GridSearchCV` , `RandomizedSearchCV` , `HalvingGridSearchCV` , `learning_curve` , `validation_curve` .

Metrics (classification): `accuracy_score` , `precision_score` , `recall_score` , `f1_score` , `roc_auc_score` , `average_precision_score` , `confusion_matrix` , `classification_report` , `roc_curve` , `precision_recall_curve` .

Metrics (regression): `mean_squared_error` , `mean_absolute_error` , `r2_score` , `mean_absolute_percentage_error` , `median_absolute_error` .

Inspection: `permutation_importance` , `PartialDependenceDisplay` , `plot_tree` .

Calibration: `CalibratedClassifierCV` , `calibration_curve` , `CalibrationDisplay` .

CHAPTER

Appendix C: Further Reading

The books, papers, and libraries I would reach for next, grouped by the chapter that motivates them. Every title here has earned the recommendation by surviving its own first impression.

C.1 Python the language

Fluent Python (Luciano Ramalho). The book to read once you're comfortable with Python and want to learn what it really offers. Concurrency, descriptors, metaclasses, generator design.

Effective Python (Brett Slatkin). 90 specific items, each a page or two. Ideal to browse in spare ten-minute windows.

C.2 NumPy and the array world

The **NumPy User Guide** and **NumPy Reference**, both on numpy.org. The User Guide's section on broadcasting and the strided memory model is better than any book I've read on either topic.

Python Data Science Handbook (Jake VanderPlas). Free online. The NumPy and pandas chapters are a lucid, example-first tour.

C.3 pandas, beyond the basics

Python for Data Analysis (Wes McKinney). Written by the original author of pandas. Worth reading cover to cover if you work with tabular data every day.

Polars documentation. Pandas-shaped API, but Rust-based and much faster on large data. A good first read once pandas becomes a bottleneck.

DuckDB documentation. Drop-in, in-process SQL over Parquet, CSV, and pandas. For the cases where SQL is clearer than pandas — and it often is.

C.4 Visualization

The Visual Display of Quantitative Information (Edward Tufte). Still the best book on what a chart should do.

Better Data Visualizations (Jonathan Schwabish). A practical catalogue of chart types with opinionated advice on when to use each.

seaborn documentation. When you want matplotlib's plots without writing matplotlib's code.

Observable Plot. Not Python, but the grammar is worth studying — Plotly's Python API (`plotly.express`) borrows from the same ideas.

C.5 Statistics

Statistical Rethinking (Richard McElreath). A Bayesian modeling textbook that you can read even if the word "Bayesian" is new to you. Uses R; the Python port (`numpyro` , `pymc`) is excellent.

An Introduction to Statistical Learning (James, Witten, Hastie, Tibshirani). Free online. The textbook version of scikit-learn's user guide. Read the first eight chapters slowly.

C.6 Machine learning

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Aurélien Géron). A thorough, practical reference. The first half tracks this book's Part V closely and goes deeper.

The Elements of Statistical Learning (Hastie, Tibshirani, Friedman). Free online. Dense; a reference more than a read-through. Most questions you will have about a scikit-learn estimator have an answer here.

Designing Machine Learning Systems (Chip Huyen). What happens after training works — data contracts, drift, experimentation, and the organizational layer.

C.7 Deep learning

Deep Learning with PyTorch (Stevens, Antiga, Viehmann). The best introduction I've read.

JAX documentation. Functional, composable, very fast. The quickstart is short; the underlying ideas take longer to internalize but reward the investment.

Dive into Deep Learning (Zhang, Lipton, Li, Smola). Free online. Same book implemented in PyTorch, MXNet, and TensorFlow. Comparison- friendly.

C.8 Engineering the surrounding code

The Pragmatic Programmer (Hunt, Thomas). The default answer for "what should I read to write better code" regardless of stack.

A Philosophy of Software Design (John Ousterhout). Short, opinionated, deeply useful. The only book I reread every year or two.

Ruff (charliermarsh/ruff). Absurdly fast Python linter and formatter, written in Rust. Replaces `flake8`, `isort`, `black`, and most plugins.

Pytest documentation. Pytest is the Python testing framework. You will eventually need to write tests; it's easier than you think.

C.9 When you want to stop reading books

arxiv-sanity. A reading list of ML papers filtered by simple heuristics. Useful for staying current without drowning.

Hacker News, the `ycombinator.com/best` sorting. The Python and data-science stories that hit the front page are usually worth at least a glance.

Conference talks. SciPy, PyCon, NeurIPS. Watch the keynotes first; pick one or two talks per conference that match your current problem.

That's the whole book. Good luck — and when the next version of scikit-learn comes out and half of chapter 15's APIs change slightly, remember that the ideas outlast the syntax.